Specifiche Tecniche: Sistema di Gestione Automatica Blacklist IP

Versione: 1.0
Data: 26 Gennaio 2026
Stato: Draft - In Revisione
Autore: Sistema di Specifica Tecnica

---

1. Executive Summary

1.1 Obiettivo

Implementare un sistema automatizzato per rilevare, gestire e recuperare IP address che vengono inseriti in blacklist da parte di mail provider esterni, garantendo:

- **Alta affidabilità**: Zero falsi negativi (nessun IP blacklistato passa inosservato)
- **Bassa latenza**: Rilevamento entro 5-15 minuti dall'evento
- **Automazione bilanciata**: Minimizzare intervento umano mantenendo controllo decisionale
- **Osservabilità completa**: Audit trail, metriche, dashboard per monitoraggio operativo

1.2 Scope

Il sistema copre l'intero ciclo di vita di un IP address nel contesto della reputazione:

1. **Detection**: Analisi in tempo reale delle risposte SMTP per identificare segnali di blacklist
2. **Quarantine**: Esclusione automatica degli IP problematici dall'uso attivo
3. **Verification**: Verifica automatica dello stato di delist tramite strategie multiple
4. **Recovery**: Reinserimento controllato degli IP nel pool attivo
5. **Notification**: Alerting multi-canale per operatori umani

1.3 Benefici Attesi

- **Protezione reputazione**: Evitare l'uso continuato di IP blacklistati che peggiorerebbe ulteriormente la reputazione
- **Deliverability ottimizzata**: Mantenere alto tasso di consegna utilizzando solo IP in buono stato
- **Riduzione carico operativo**: Automazione delle verifiche e azioni ripetitive
- **Visibilità proattiva**: Dashboard e alert permettono intervento preventivo
- **Audit e compliance**: Tracciabilità completa di tutte le decisioni e azioni

1.4 Impatto e Rischi Mitigati

Rischi Attuali (senza sistema):

- IP blacklistati continuano a inviare, peggiorando la situazione
- Detection manuale lenta (ore/giorni)
- Nessuna traccia storica delle problematiche
- Impossibilità di gestire volumi elevati

Mitigazioni Offerte:

- Detection automatica in 5-15 minuti
- Quarantena immediata con enforcement automatico
- Audit trail completo per analisi post-mortem
- Scalabilità per gestire migliaia di segnali/ora

---

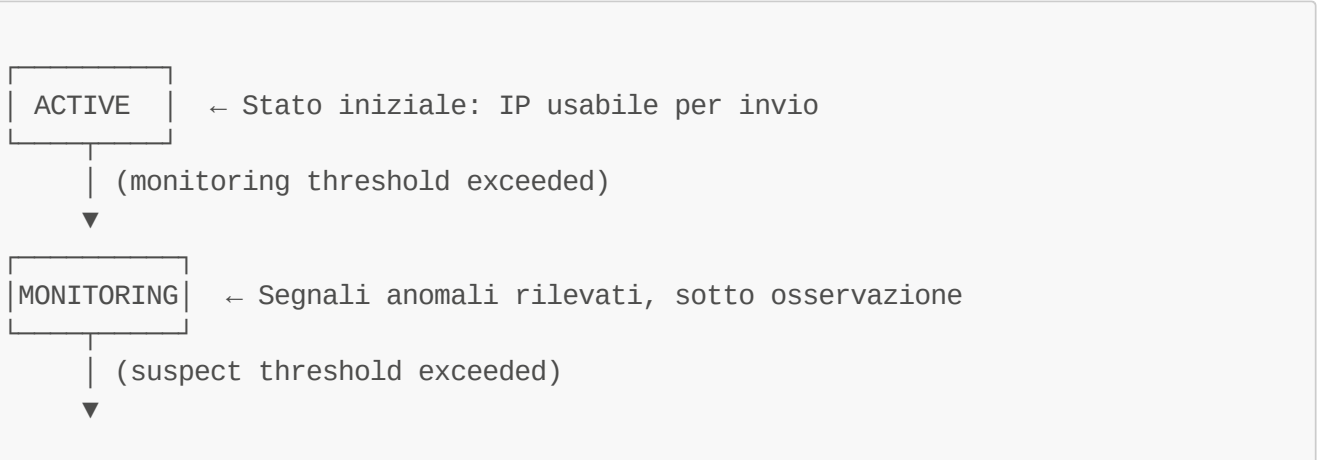
2. Glossario e Definizioni

2.1 Terminologia

Termine	Definizione
Blacklist	Lista di IP address bloccati da un mail provider per invio spam, phishing o altre violazioni delle policy
RBL (Realtime Blackhole List)	Database DNS-based di IP blacklistati, consultabile pubblicamente (es. Spamhaus, SORBS)
Greylist	Tecnica temporanea di deferral delle email; non è una blacklist permanente
Reputation	Punteggio di affidabilità di un IP basato su comportamento storico di invio
Signal	Evento grezzo estratto da una risposta SMTP che indica possibile blacklist
Evidence	Aggregazione di più signal che supporta una decisione di cambio stato
Confidence Score	Valore 0-1 che rappresenta la certezza che un IP sia blacklistato
Quarantine	Stato di esclusione temporanea di un IP dal pool attivo
Delist	Processo di rimozione di un IP da una blacklist (naturale o tramite richiesta)
Probe	Tentativo controllato di verifica dello stato di un IP (invio test o query RBL)
Pool Health	Percentuale di IP attivi rispetto al totale del pool
Flapping	Oscillazione rapida tra stati (es. ACTIVE ↔ QUARANTINED) da evitare

2.2 Stati del Ciclo di Vita IP

Un IP address può trovarsi in uno dei seguenti stati:



```
┌─────────────┐
│ SUSPECT │  ← Segnali significativi, probabile blacklist
└──────┬──────┘
       │ (quarantine threshold exceeded)
       ▼
┌──────────────┐
│QUARANTINED │  ← IP escluso dall'uso, in attesa di verifica
└──────┬───────┘
       │ (delist check scheduled)
       ▼
┌──────────────┐
│DELIST_CHECK │  ← Verifica in corso dello stato blacklist
└──────┬───────┘
       │
       ├─ (verification successful) ──→ ACTIVE
       │
       ├─ (verification failed) ──→ QUARANTINED (backoff)
       │
       └─ (verification inconclusive) ──→ MANUAL_REVIEW
                                                 │
                                                 └─ (human decision) ──→ ACTIVE |
QUARANTINED
```

2.3 Definizione degli Stati

ACTIVE

- **Descrizione**: IP in pieno utilizzo, nessun segnale di problemi
- **Comportamento**: Selezionabile da `IPAddress.select_by_priority`
- **Transizione a MONITORING**: Rilevati segnali anomali (sotto soglia critica)

MONITORING

- **Descrizione**: IP ancora usabile ma sotto osservazione aumentata
- **Comportamento**: Selezionabile, ma con logging dettagliato e confidence tracking
- **Transizione a SUSPECT**: Accumulo di segnali blacklist (confidence > 0.70)
- **Transizione a ACTIVE**: Segnali rientrati, periodo di osservazione positivo

SUSPECT

- **Descrizione**: Forte sospetto di blacklist, uso limitato consigliato
- **Comportamento**: Selezionabile solo se nessun altro IP disponibile (degraded mode)
- **Transizione a QUARANTINED**: Soglia critica superata (confidence > 0.80)
- **Transizione a MONITORING**: Segnali diminuiti, possibile falso positivo

QUARANTINED

- **Descrizione**: IP confermato o altamente sospetto di blacklist, non utilizzabile
- **Comportamento**: **Escluso** da selezione automatica, nessun invio
- **Durata minima**: Configurabile (default: 24h)

- **Transizione a DELIST_CHECK**: Dopo durata minima, inizia verifica automatica
- **Transizione manuale a ACTIVE**: Solo tramite override operatore (logged)

DELIST_CHECK

- **Descrizione**: Verifica automatica in corso per determinare se IP è stato delistato
- **Comportamento**: Non utilizzabile, in attesa risultato probe/query
- **Transizione a ACTIVE**: Verifica positiva (IP delistato con successo)
- **Transizione a QUARANTINED**: Verifica negativa (IP ancora blacklistato) con backoff
- **Transizione a MANUAL_REVIEW**: Verifica inconclusiva o errori

MANUAL_REVIEW

- **Descrizione**: Sistema non può determinare stato, richiede intervento umano
- **Comportamento**: Non utilizzabile, alert critico generato
- **Transizione**: Solo manuale da operatore con motivazione obbligatoria

2.4 Tipi di Signal

Signal Type	Descrizione	Esempi SMTP	Confidence Base
HARD_BLACKLIST	Menzione esplicita di RBL o block permanente	`550 blocked using zen.spamhaus.org`	0.95
SOFT_BLACKLIST	Rate limiting o reputazione bassa IP-specific	`421 too many connections from your IP`	0.70
REPUTATION_ISSUE	Segnali di spam score o complaints	`550 sender reputation too low`	0.75
GREYLIST	Deferral temporaneo (non è blacklist)	`451 Greylisted, try again later`	0.10
UNKNOWN	Pattern non classificabile, da investigare	Messaggi ambigui	0.50

2.5 Metriche Chiave

- **Error Rate**: Percentuale di delivery failure (HardFail + SoftFail) su totale tentativi
- **Confidence Score**: Probabilità aggregata che un IP sia blacklistato (0.0 - 1.0)
- **Signal Density**: Numero di signal per unità di tempo (signal/min)
- **Provider Diversity**: Numero di provider distinti che segnalano problemi
- **Pool Health Ratio**: IP attivi / IP totali nel pool
- **Quarantine Duration**: Tempo medio di permanenza in quarantena
- **False Positive Rate**: IP quarantined erroneamente / totale quarantine
- **Delist Success Rate**: Verifiche delist riuscite / totale verifiche

---

3. Architettura Proposta

3.1 Overview

Il sistema estende Postal con 5 componenti principali che lavorano in pipeline:

1. **Signal Capture** (DetectorService) - Analizza risposte SMTP real-time
2. **Aggregation & Analysis** (MonitorService) - Aggrega signals, calcola confidence
3. **State Management** (QuarantineService) - Gestisce transizioni e enforcement
4. **Delist Verification** (DelistVerifierService) - Verifica automatica recovery
5. **Notification** (NotificationService) - Alert multi-canale

3.2 Database Schema

6 nuove tabelle integrate nel database Postal:

Tabella	Scopo	Volume	Retention
`ip_address_statuses`	Stato corrente ogni IP	Low (1 per IP)	Permanente
`ip_blacklist_signals`	Eventi grezzi SMTP	High (1000s/hour)	90 giorni
`ip_blacklist_patterns`	Config pattern detection	Low (~50 records)	Permanente
`ip_blacklist_evidence`	Aggregazioni analitiche	Medium (~100s/day)	180 giorni
`ip_probe_attempts`	Log verifiche delist	Low (~10s/day)	90 giorni
`ip_blacklist_audit_logs`	Compliance audit trail	Low (~20s/day)	2 anni

Relazioni chiave:

- `IPAddress` 1:1 `IPAddressStatus`
- `IPAddress` 1:N `IPBlacklistSignal`, `IPBlacklistEvidence`, `IPProbeAttempt`

3.3 Integration Points

Hook in Delivery Flow:

```
# app/lib/message_dequeuer/outgoing_message_processor.rb
after_delivery :capture_blacklist_signal

def capture_blacklist_signal
  return unless @result.type.in?(["HardFail", "SoftFail"])
  IPBlacklist::DetectorService.call(...)
end
```

IP Selection Override:

```
# app/models/ip_address.rb
scope :usable_for_sending, -> {
  left_joins(:blacklist_status)
```

```
    .where("status IN (?) OR status IS NULL", ["active", "monitoring"])
}
```

Background Jobs:

- `IPBlacklistMonitorJob` - ogni 5 min (aggregazione)
- `IPDelistCheckJob` - ogni 1h (verifiche)
- `IPBlacklistCleanupJob` - daily (pruning)

3.4 Flusso End-to-End

```
SMTP Error (550 RBL) → DetectorService → Signal Created
  ↓ (5 min)
MonitorService → Aggrega Signals → Calcola Confidence → Evidence
  ↓
QuarantineService → Transizione Stato → Enforcement → Notification
  ↓ (24h min quarantine)
DelistVerifierService → Probe Strategies → Success/Fail
  ↓
Return to ACTIVE o Extended Quarantine
```

Dettagli implementativi completi: Vedi sezioni 6 (Quarantine), 7 (Delist), 9 (Data Model), 10 (Components).

---

4. Detection Logic - Raccolta Segnali

4.1 Principio di Funzionamento

Ogni delivery con stato `HardFail` o `SoftFail` viene analizzata in real-time dal **DetectorService** che:

1. Applica pattern matching su codice SMTP e messaggio
2. Classifica il tipo di errore (blacklist vs recipient invalid vs quota)
3. Calcola confidence score basato su pattern matched
4. Crea record `IPBlacklistSignal` se rilevante

4.2 Pattern Matching Engine

Configurabile via tabella `ip_blacklist_patterns`:

Pattern Type	Esempio Regex Message	Confidence
HARD_BLACKLIST	`(zen\.spamhaus\.org|spamhaus)`	0.95
SOFT_BLACKLIST	`too many.*from.*IP`	0.70
REPUTATION_ISSUE	`IP.*reputation.*low`	0.75
EXCLUSION	`recipient.*not found`	0.0 (skip)

Pattern predefiniti inclusi per:

- RBL major: Spamhaus, Barracuda, SpamCop, SORBS
- IP-specific blocks
- Rate limiting IP-based
- Reputation issues
- Exclusion patterns (recipient errors, quota, content filters)

Supporto multi-lingua: Pattern con varianti IT, FR, DE, ES per messaggi comuni.

4.3 Signal Enrichment

Ogni signal include metadata:

- Pattern matched (ID + name)
- Provider domain
- SMTP code + enhanced code
- RBL names detected (extracted from message)
- Confidence score
- Timestamp precise

4.4 False Positive Mitigation

- **Exclusion patterns**: Errori non-IP esplicitamente esclusi
- **Context awareness**: Distingue spike da pattern persistente
- **Feedback loop**: Operatori possono mark signals come false positive
- **Confidence decay**: Segnali vecchi pesano meno

Performance: Signal capture < 50ms, non-blocking, circuit breaker su errori.

Dettagli implementativi completi: Vedi sezione 10.1 (DetectorService code).

---

5. Decisional Logic - Aggregazione e Scoring

5.1 Time Windows

Il sistema analizza signals su 3 finestre temporali:

Window	Durata	Scopo
Short	15 min	Detection rapida blacklist acute
Medium	1h	Trend consolidamento
Long	24h	Valutazione reputazione globale

5.2 Confidence Score Algorithm

Formula base:

```
confidence = weighted_avg(signals) × diversity_boost × recency_weight × (1 -
```

```
noise_penalty)
```

Componenti:

1. **Weighted Average**: Signals pesati per tipo (hard=1.0, soft=0.7, reputation=0.5)
2. **Diversity Boost**:
   - 1 provider → 1.0x
   - 2 provider → 1.15x
   - 3 provider → 1.30x
   - 4+ provider → 1.50x
3. **Recency Weight**: Decay esponenziale, segnali recenti pesano di più
4. **Noise Penalty**: Max 20% penalty per segnali "unknown" o low-confidence

Esempio:

- 3 hard signals (conf 0.95) + 1 soft (0.70) da 3 provider diversi in 10 min
- Base: 0.88, Diversity: ×1.30, Recency: ×0.95, Noise: -0
- **Final confidence: 1.09 → capped a 1.0**

5.3 State Transition Thresholds

From → To	Condizioni
ACTIVE → MONITORING	min_signals=1, confidence≥0.50, cooldown=5min
MONITORING → SUSPECT	min_signals=2, hard≥1, confidence≥0.70, cooldown=10min
SUSPECT → QUARANTINED	min_signals=3, hard≥2, confidence≥0.80, providers≥2, cooldown=30min
QUARANTINED → DELIST_CHECK	min_duration=24h

Tutte le soglie sono configurabili via YAML config.

5.4 Anti-Flapping Mechanisms

- **Cooldown periods**: Minimo tempo tra transizioni stesso IP
- **Hysteresis**: Soglie diverse per salita (0.80) e discesa (0.50)
- **Consecutive confirmation**: Certe transizioni richiedono conferma su 2 cicli
- **Exponential backoff**: Re-quarantine usa durata crescente (1.5x)

5.5 MonitorService Flow

```
ogni 5 minuti:
  per ogni IP con signals recenti:
    raccogliere signals(short, medium, long windows)
    calcolare confidence(short, medium, long)
    determinare raccomandazione stato
    se raccomandazione != stato corrente:
```

```
      if cooldown_elapsed:
        eseguire transizione
        creare evidence record
        notificare
```

Dettagli implementativi completi: Vedi sezione 10.2 (MonitorService code) e 6 (QuarantineService).

---

6. Quarantine Strategy

6.1 Obiettivi Quarantine

1. **Esclusione immediata** dell'IP dal pool di selezione
2. **Durata configurabile** basata su severity e storico
3. **Pool health monitoring** per evitare degradazione servizio
4. **Backoff esponenziale** per IP recidivi
5. **Override manuale** tracciato per emergenze

6.2 Enforcement Mechanism

6.2.1 Modifica IP Selection

File: `app/models/ip_address.rb`

```ruby
class IPAddress < ApplicationRecord

  scope :usable_for_sending, -> {
    left_joins(:blacklist_status)
      .where(
        "ip_address_statuses.id IS NULL OR
         ip_address_statuses.status IN (?)",
        IPAddressStatus::USABLE_STATES
      )
  }

  def self.select_by_priority
    candidates = usable_for_sending.order("RAND() * priority DESC")

    # Pool health check
    if candidates.count < minimum_active_ips
      handle_insufficient_ips(candidates)
    else
      candidates.first
    end
  end

  def self.minimum_active_ips
    Rails.application.config.ip_blacklist.pool_health.min_active_ips
  end

  private
```

```
  def self.handle_insufficient_ips(candidates)
    # Log alert critico
    KLogger.error("IP pool health critical", {
      available: candidates.count,
      minimum: minimum_active_ips,
      pool_id: first&.ip_pool_id
    })

    # Notifica immediata
    IPBlacklist::NotificationService.call(
      event: "pool_health_critical",
      severity: "critical",
      data: { pool_id: first&.ip_pool_id, available: candidates.count }
    )

    # Fallback: usa anche IP SUSPECT se disponibili (degraded mode)
    fallback_candidates = where(
      blacklist_status: { status: ["active", "monitoring", "suspect"] }
    )

    if fallback_candidates.any?
      KLogger.warn("Using SUSPECT IPs in degraded mode")
      fallback_candidates.order("RAND() * priority DESC").first
    else
      # Emergenza: ritorna nil, invio fallirà con held
      nil
    end
  end

end
```

6.2.2 IPAddressStatus Constants

```
class IPAddressStatus < ApplicationRecord
  STATES = %w[
    active
    monitoring
    suspect
    quarantined
    delist_check
    manual_review
  ].freeze

  USABLE_STATES = %w[active monitoring].freeze
  DEGRADED_USABLE = %w[active monitoring suspect].freeze
  NON_USABLE = %w[quarantined delist_check manual_review].freeze

  validates :status, inclusion: { in: STATES }

  def quarantined?
```

```
    status == "quarantined"
  end

  def usable?
    status.in?(USABLE_STATES)
  end
end
```

6.3 Quarantine Duration Logic

6.3.1 Base Duration Calculation

```
module IPBlacklist
  class QuarantineDurationCalculator

    def self.calculate(ip_address, evidence)
      new(ip_address, evidence).calculate
    end

    def calculate
      base = base_duration
      multiplier = backoff_multiplier
      severity_factor = severity_adjustment

      final_duration = base * multiplier * severity_factor

      # Cap massimo configurabile
      [final_duration, max_duration].min
    end

    private

    def base_duration
      config.quarantine.initial_duration  # default: 24.hours
    end

    def backoff_multiplier
      # Exponential backoff per IP recidivi
      quarantine_count = @ip_address.blacklist_status&.quarantine_count || 0
      multiplier = config.quarantine.backoff_multiplier  # default: 1.5

      [multiplier ** quarantine_count, 10.0].min
    end

    def severity_adjustment
      # Adjust based on evidence severity
      case @evidence&.evidence_type
      when "critical_rbl"
        2.0   # RBL major (Spamhaus) → 2x duration
      when "hard_blacklist"
        1.5   # Hard blacklist generico → 1.5x
```

```
        when "soft_blacklist"
          1.0   # Soft blacklist → base
        when "reputation_issue"
          0.75  # Solo reputation → 0.75x
        else
          1.0
        end
      end

      def max_duration
        config.quarantine.max_duration  # default: 7.days
      end

      def config
        Rails.application.config.ip_blacklist
      end

    end
  end
```

6.3.2 Example Durations

Quarantine #	Base	Multiplier	Severity	Final Duration
1st (RBL)	24h	1.0	2.0	**48h**
2nd (RBL)	24h	1.5	2.0	**72h**
3rd (RBL)	24h	2.25	2.0	**108h (4.5d)**
4th (RBL)	24h	3.375	2.0	**162h (6.75d)**
1st (soft)	24h	1.0	1.0	**24h**
2nd (soft)	24h	1.5	1.0	**36h**

6.4 QuarantineService Implementation

```
module IPBlacklist
  class QuarantineService

    def self.transition_to(ip_address:, new_state:, evidence:, actor:, reason:
nil)
      new(ip_address, new_state, evidence, actor, reason).execute
    end

    def execute
      ActiveRecord::Base.transaction do
        # 1. Verifica cooldown
        return false unless cooldown_elapsed?

        # 2. Update o crea status record
```

```
        update_status

        # 3. Calcola duration se quarantine
        calculate_quarantine_end if @new_state == "quarantined"

        # 4. Crea audit log
        create_audit_log

        # 5. Trigger notification
        notify_transition

        # 6. Update metrics
        update_metrics
      end

      true
    end

    private

    def update_status
      status = @ip_address.blacklist_status ||
@ip_address.build_blacklist_status

      old_state = status.status

      status.update!(
        status: @new_state,
        status_changed_at: Time.current,
        confidence_score: @evidence&.aggregate_confidence || 0.0,
        quarantine_count: increment_quarantine_count(status),
        last_quarantined_at: (@new_state == "quarantined" ? Time.current :
status.last_quarantined_at),
        metadata: status.metadata.merge(
          last_evidence_id: @evidence&.id,
          last_transition_actor: @actor,
          last_transition_reason: @reason
        )
      )

      @old_state = old_state
      status
    end

    def increment_quarantine_count(status)
      if @new_state == "quarantined"
        (status.quarantine_count || 0) + 1
      else
        status.quarantine_count || 0
      end
    end

    def calculate_quarantine_end
      duration = QuarantineDurationCalculator.calculate(@ip_address, @evidence)
```

```
    @quarantine_end = Time.current + duration

    @ip_address.blacklist_status.update!(
      metadata: @ip_address.blacklist_status.metadata.merge(
        quarantine_duration_seconds: duration.to_i,
        quarantine_until: @quarantine_end,
        backoff_applied: duration > config.quarantine.initial_duration
      )
    )
  end

  def create_audit_log
    IPBlacklistAuditLog.create!(
      ip_address: @ip_address,
      event_type: "state_transition",
      from_status: @old_state,
      to_status: @new_state,
      actor_id: actor_id,
      actor_type: @actor == "System" ? "System" : "User",
      reason: @reason || "Automatic transition based on evidence",
      evidence_snapshot: evidence_snapshot,
      occurred_at: Time.current
    )
  end

  def evidence_snapshot
    return {} unless @evidence

    {
      evidence_id: @evidence.id,
      confidence: @evidence.aggregate_confidence,
      signal_count: @evidence.signal_count,
      providers: @evidence.providers_affected,
      patterns: @evidence.patterns_matched,
      window: "#{@evidence.window_start} - #{@evidence.window_end}"
    }
  end

  def notify_transition
    IPBlacklist::NotificationService.call(
      event: "ip_state_changed",
      severity: notification_severity,
      data: notification_data
    )
  end

  def notification_severity
    case @new_state
    when "monitoring" then "info"
    when "suspect" then "warning"
    when "quarantined" then "error"
    when "manual_review" then "critical"
    when "active" then "info"
```

```
      else "warning"
      end
    end

    def notification_data
      {
        ip_address: @ip_address.ipv4 || @ip_address.ipv6,
        ip_address_id: @ip_address.id,
        pool_name: @ip_address.ip_pool&.name,
        old_state: @old_state,
        new_state: @new_state,
        confidence: @evidence&.aggregate_confidence,
        evidence_id: @evidence&.id,
        quarantine_until: @quarantine_end,
        actor: @actor,
        reason: @reason
      }
    end

  end
end
```

6.5 Pool Health Management

6.5.1 Health Metrics

```
class IPPool < ApplicationRecord

  def health_metrics
    total = ip_addresses.count
    active = ip_addresses.joins(:blacklist_status)
                         .where(ip_address_statuses: { status: "active"
}).count
    monitoring = ip_addresses.joins(:blacklist_status)
                             .where(ip_address_statuses: { status: "monitoring"
}).count
    quarantined = ip_addresses.joins(:blacklist_status)
                              .where(ip_address_statuses: { status:
"quarantined" }).count

    usable = active + monitoring
    health_ratio = total > 0 ? usable.to_f / total : 0.0

    {
      total: total,
      active: active,
      monitoring: monitoring,
      quarantined: quarantined,
      usable: usable,
      health_ratio: health_ratio,
      status: health_status(health_ratio)
```

```
    }
  end

  def health_status(ratio)
    case ratio
    when 0.8..1.0 then "healthy"
    when 0.5..0.8 then "degraded"
    when 0.3..0.5 then "critical"
    else "emergency"
    end
  end

  def critical_health?
    metrics = health_metrics
    config = Rails.application.config.ip_blacklist.pool_health

    metrics[:health_ratio] < config.critical_threshold ||
    metrics[:usable] < config.min_active_ips
  end

end
```

6.5.2 Automated Pool Health Checks

```
# Background job
class IPPoolHealthCheckJob
  def perform
    IPPool.find_each do |pool|
      check_pool_health(pool)
    end
  end

  private

  def check_pool_health(pool)
    metrics = pool.health_metrics

    case metrics[:status]
    when "critical", "emergency"
      IPBlacklist::NotificationService.call(
        event: "pool_health_critical",
        severity: "critical",
        data: {
          pool_id: pool.id,
          pool_name: pool.name,
          metrics: metrics,
          suggested_action: "Review quarantined IPs for manual override"
        }
      )

    when "degraded"
```

```
      IPBlacklist::NotificationService.call(
        event: "pool_health_degraded",
        severity: "warning",
        data: { pool_id: pool.id, metrics: metrics }
      )
    end
  end
end
```

6.6 Manual Override

Operatori possono forzare cambio stato per emergenze:

```
# Controller action
class IPAddressesController < ApplicationController
  def force_state_change
    @ip_address = IPAddress.find(params[:id])
    authorize @ip_address, :force_quarantine?

    new_state = params[:new_state]
    reason = params[:reason]

    unless reason.present?
      flash[:error] = "Reason is required for manual state change"
      return redirect_back(fallback_location: ip_address_path(@ip_address))
    end

    success = IPBlacklist::QuarantineService.transition_to(
      ip_address: @ip_address,
      new_state: new_state,
      evidence: nil,
      actor: current_user,
      reason: "MANUAL OVERRIDE: #{reason}"
    )

    if success
      flash[:success] = "IP state changed to #{new_state}"
    else
      flash[:error] = "Failed to change IP state"
    end

    redirect_to ip_address_path(@ip_address)
  end
end
```

---

7. Delist Verification Strategy

7.1 Obiettivi Delist Verification

1. **Verificare automaticamente** se un IP è stato rimosso da blacklist
2. **Strategie multiple** con fallback chain
3. **Minimizzare falsi negativi** (non dichiarare delistato se ancora bloccato)
4. **Tracciabilità** di ogni tentativo di verifica
5. **Gestire inconclusivi** con escalation a manual review

7.2 Strategy Chain

Il sistema usa 3 strategie in ordine di priorità:

```
1. PRIMARY: Retry Original Message
   ↓ (not available)
2. SECONDARY: DNS RBL Queries
   ↓ (RBL not queryable or timeout)
3. TERTIARY: Time-Based Expiry
```

7.3 Strategy 1: Retry Original Message

Rationale: Il modo più affidabile per verificare se un IP è delistato è provare a re-inviare lo stesso messaggio che aveva causato l'errore.

7.3.1 Implementation

```ruby
module IPBlacklist
  module DelistStrategies
    class RetryOriginalMessage

      def self.call(ip_address)
        new(ip_address).execute
      end

      def execute
        # 1. Trova ultimo messaggio fallito per questo IP
        failed_delivery = find_last_failed_delivery

        return inconclusive("No failed delivery found") unless failed_delivery

        # 2. Verifica che il messaggio sia ancora in coda o recuperabile
        queued_message = find_or_recreate_queued_message(failed_delivery)

        return inconclusive("Message no longer available") unless
queued_message

        # 3. Forza invio tramite questo specifico IP
        result = send_via_specific_ip(queued_message, @ip_address)

        # 4. Analizza risultato
        evaluate_result(result, failed_delivery)
      end
```

```
      private

      def find_last_failed_delivery
        # Query MessageDB per ultimo delivery fallito
        @ip_address.server.message_db.database.query(
          "SELECT * FROM deliveries
           WHERE ip_address_id = ?
           AND status IN ('HardFail', 'SoftFail')
           ORDER BY timestamp DESC
           LIMIT 1",
          @ip_address.id
        ).first
      end

      def find_or_recreate_queued_message(delivery)
        message = @ip_address.server.message_db.message(delivery.message_id)

        # Crea nuovo QueuedMessage per retry
        QueuedMessage.create!(
          server: @ip_address.server,
          message_id: message.id,
          ip_address: @ip_address,  # FORZA questo IP
          locked_by: "delist_probe",
          locked_at: Time.current,
          manual: true,  # bypass normal allocation
          retry_after: Time.current
        )
      rescue => e
        KLogger.error("Failed to recreate queued message: #{e.message}")
        nil
      end

      def send_via_specific_ip(queued_message, ip_address)
        # Usa OutgoingMessageProcessor ma forza IP allocation
        processor =
MessageDequeuer::OutgoingMessageProcessor.new(queued_message)

        # Override IP già allocato
        result = processor.process

        {
          status: result.type,
          code: result.output&.match(/^(\d{3})/)[1],
          message: result.output,
          delivery_id: processor.delivery&.id
        }
      rescue => e
        {
          status: "Error",
          error: e.message
        }
      end
```

```
      def evaluate_result(result, original_delivery)
        case result[:status]
        when "Sent"
          # SUCCESS: IP delistato!
          create_probe_record(
            status: "success",
            result_details: "Message delivered successfully, IP appears
delisted"
          )

          :success

        when "HardFail", "SoftFail"
          # Analizza se stesso errore blacklist
          if same_blacklist_error?(result, original_delivery)
            create_probe_record(
              status: "failed",
              result_details: "Same blacklist error detected: #
{result[:message]}"
            )

            :failed
          else
            # Errore diverso (recipient invalid, quota, ecc.) → inconclusive
            create_probe_record(
              status: "inconclusive",
              result_details: "Different error: #{result[:message]}"
            )

            :inconclusive
          end

        else
          # Errore tecnico
          create_probe_record(
            status: "error",
            result_details: "Technical error: #{result[:error]}"
          )

          :error
        end
      end

      def same_blacklist_error?(current_result, original_delivery)
        # Ri-applica pattern detection
        detector = IPBlacklist::DetectorService.new(
          smtp_code: current_result[:code],
          smtp_message: current_result[:message],
          # ... altri params
        )

        signal = detector.call

        # Se rileva stesso tipo di blacklist signal → ancora blacklistato
```

```
        signal&.signal_type == "hard_blacklist" || signal&.signal_type ==
"soft_blacklist"
      end

      def create_probe_record(status:, result_details:)
        IPProbeAttempt.create!(
          ip_address: @ip_address,
          probe_type: "retry_message",
          status: status,
          result_details: result_details,
          attempted_at: Time.current,
          completed_at: Time.current,
          metadata: {
            delivery_id: @delivery_id,
            strategy: "retry_original_message"
          }
        )
      end

      def inconclusive(reason)
        create_probe_record(
          status: "inconclusive",
          result_details: reason
        )

        :inconclusive
      end

    end
  end
end
```

7.4 Strategy 2: DNS RBL Queries

Rationale: Verificare direttamente presso RBL pubblici se l'IP è ancora listato.

7.4.1 Implementation

```
module IPBlacklist
  module DelistStrategies
    class RBLQuery

      SUPPORTED_RBLS = [
        "zen.spamhaus.org",
        "bl.spamcop.net",
        "b.barracudacentral.org",
        "dnsbl.sorbs.net"
      ].freeze

      def self.call(ip_address, rbl_names: nil)
        new(ip_address, rbl_names).execute
```

```
        end

        def execute
          # 1. Determina quali RBL controllare
          rbls_to_check = determine_rbls

          return inconclusive("No RBL to query") if rbls_to_check.empty?

          # 2. Query ogni RBL
          results = query_rbls(rbls_to_check)

          # 3. Valuta risultati aggregati
          evaluate_results(results)
        end

        private

        def determine_rbls
          # Usa RBL specificati o tutti quelli menzionati nei signals
          if @rbl_names.present?
            @rbl_names
          else
            # Estrai da signals storici
            signals = @ip_address.blacklist_signals
                                 .where("metadata->>'rbl_detected' IS NOT NULL")
                                 .limit(10)

            rbls = signals.flat_map { |s| s.metadata["rbl_detected"]
}.compact.uniq

            # Filtra solo quelli supportati
            rbls & SUPPORTED_RBLS
          end
        end

        def query_rbls(rbls)
          rbls.map do |rbl|
            query_single_rbl(rbl)
          end
        end

        def query_single_rbl(rbl)
          # Reverse IP per DNS query
          reversed_ip = reverse_ip(@ip_address.ipv4)
          query_host = "#{reversed_ip}.#{rbl}"

          begin
            Timeout.timeout(5) do
              result = Resolv::DNS.new.getaddress(query_host)

              # Se risolve → IP ancora listato
              {
                rbl: rbl,
                listed: true,
```

```
            response: result.to_s,
            error: nil
          }
        end
      rescue Resolv::ResolvError => e
        # NXDOMAIN → IP NON listato (buono!)
        {
          rbl: rbl,
          listed: false,
          response: "NXDOMAIN",
          error: nil
        }
      rescue => e
        # Errore query (timeout, network)
        {
          rbl: rbl,
          listed: nil,
          response: nil,
          error: e.message
        }
      end
    end

    def reverse_ip(ip)
      ip.split('.').reverse.join('.')
    end

    def evaluate_results(results)
      listed_count = results.count { |r| r[:listed] == true }
      not_listed_count = results.count { |r| r[:listed] == false }
      error_count = results.count { |r| r[:listed].nil? }

      status, result_details = if listed_count > 0
        # Ancora listato in almeno 1 RBL → FAILED
        [
          "failed",
          "Still listed in #{listed_count} RBL(s): #{results.select { |r|
r[:listed] }.map { |r| r[:rbl] }.join(', ')}"
        ]
      elsif not_listed_count == results.size
        # Non listato in nessun RBL → SUCCESS
        [
          "success",
          "Not listed in any queried RBL (#{not_listed_count} checked)"
        ]
      elsif error_count == results.size
        # Tutti errori → ERROR
        [
          "error",
          "All RBL queries failed"
        ]
      else
        # Mix risultati → INCONCLUSIVE
        [
```

```
            "inconclusive",
            "Mixed results: #{not_listed_count} clean, #{error_count} errors"
          ]
        end

        create_probe_record(status, result_details, results)

        status.to_sym
      end

      def create_probe_record(status, result_details, rbl_results)
        IPProbeAttempt.create!(
          ip_address: @ip_address,
          probe_type: "rbl_query",
          status: status,
          result_details: result_details,
          attempted_at: Time.current,
          completed_at: Time.current,
          metadata: {
            strategy: "dns_rbl_query",
            rbls_queried: rbl_results.map { |r| r[:rbl] },
            rbl_results: rbl_results
          }
        )
      end

      def inconclusive(reason)
        create_probe_record("inconclusive", reason, [])
        :inconclusive
      end

    end
  end
end
```

7.5 Strategy 3: Time-Based Expiry

Rationale: Se nessuna verifica attiva è possibile, assumere delist dopo periodo configurabile.

```
module IPBlacklist
  module DelistStrategies
    class TimeBasedExpiry

      def self.call(ip_address)
        new(ip_address).execute
      end

      def execute
        status = @ip_address.blacklist_status

        quarantine_duration = Time.current - status.last_quarantined_at
```

```
        expiry_threshold = config.delist.time_based_threshold  # default:
7.days

        if quarantine_duration >= expiry_threshold
          create_probe_record(
            status: "success",
            result_details: "Time-based expiry: #{quarantine_duration.to_i /
86400} days elapsed"
          )

          :success
        else
          create_probe_record(
            status: "pending",
            result_details: "Waiting for time threshold (#{(expiry_threshold -
quarantine_duration).to_i / 86400} days remaining)"
          )

          :pending
        end
      end

      private

      def create_probe_record(status, result_details)
        IPProbeAttempt.create!(
          ip_address: @ip_address,
          probe_type: "time_based",
          status: status,
          result_details: result_details,
          attempted_at: Time.current,
          completed_at: Time.current,
          metadata: {
            strategy: "time_based_expiry",
            quarantine_duration_days: (Time.current -
@ip_address.blacklist_status.last_quarantined_at) / 1.day
          }
        )
      end

    end
  end
end
```

7.6 DelistVerifierService (Orchestrator)

```
module IPBlacklist
  class DelistVerifierService

    def self.call
      new.call
```

```
    end

    def call
      # Trova tutti gli IP in DELIST_CHECK
      ips_to_verify = IPAddress.joins(:blacklist_status)
                               .where(ip_address_statuses: { status:
"delist_check" })

      ips_to_verify.find_each do |ip_address|
        verify_ip(ip_address)
      end
    end

    private

    def verify_ip(ip_address)
      KLogger.info("Starting delist verification for IP #{ip_address.ipv4}")

      # Strategy chain
      result = try_retry_message(ip_address) ||
               try_rbl_query(ip_address) ||
               try_time_based(ip_address)

      # Processa risultato
      handle_verification_result(ip_address, result)
    end

    def try_retry_message(ip_address)
      return nil unless config.delist.retry_message_enabled

      result = DelistStrategies::RetryOriginalMessage.call(ip_address)

      result unless result == :inconclusive || result == :error
    end

    def try_rbl_query(ip_address)
      return nil unless config.delist.rbl_query_enabled

      result = DelistStrategies::RBLQuery.call(ip_address)

      result unless result == :inconclusive || result == :error
    end

    def try_time_based(ip_address)
      return nil unless config.delist.time_based_fallback

      DelistStrategies::TimeBasedExpiry.call(ip_address)
    end

    def handle_verification_result(ip_address, result)
      case result
      when :success
        # Delist successful → ACTIVE
        transition_to_active(ip_address)
```

```
      when :failed
        # Ancora blacklistato → QUARANTINED con backoff
        transition_to_quarantined_with_backoff(ip_address)

      when :inconclusive, :error
        # Non possiamo determinare → MANUAL_REVIEW
        handle_inconclusive(ip_address)

      when :pending
        # Time-based non ancora scaduto → mantieni DELIST_CHECK
        nil
      end
    end

    def transition_to_active(ip_address)
      IPBlacklist::QuarantineService.transition_to(
        ip_address: ip_address,
        new_state: "monitoring",  # Non ACTIVE diretto, ma MONITORING per
sicurezza
        evidence: nil,
        actor: "System",
        reason: "Delist verification successful"
      )

      IPBlacklist::NotificationService.call(
        event: "ip_delisted",
        severity: "info",
        data: {
          ip_address: ip_address.ipv4,
          ip_address_id: ip_address.id,
          quarantine_duration: (Time.current -
ip_address.blacklist_status.last_quarantined_at).to_i / 3600
        }
      )
    end

    def transition_to_quarantined_with_backoff(ip_address)
      IPBlacklist::QuarantineService.transition_to(
        ip_address: ip_address,
        new_state: "quarantined",
        evidence: nil,
        actor: "System",
        reason: "Delist verification failed, IP still blacklisted"
      )

      # Backoff automatico applicato da QuarantineDurationCalculator
    end

    def handle_inconclusive(ip_address)
      # Conta tentativi inconclusivi
      inconclusive_count = ip_address.probe_attempts
                                     .where(status: ["inconclusive", "error"])
                                     .where("attempted_at > ?", 7.days.ago)
```

```
                                        .count

      if inconclusive_count >=
config.thresholds.manual_review.inconclusive_probes
        # Troppi inconclusivi → MANUAL_REVIEW
        IPBlacklist::QuarantineService.transition_to(
          ip_address: ip_address,
          new_state: "manual_review",
          evidence: nil,
          actor: "System",
          reason: "Unable to verify delist after #{inconclusive_count}
attempts"
        )
      else
        # Riprova più tardi
        schedule_retry(ip_address, delay: 6.hours)
      end
    end

    def config
      Rails.application.config.ip_blacklist
    end

  end
end
```

7.7 Scheduled Job

```
# app/lib/worker/jobs/ip_delist_check_job.rb
class IPDelistCheckJob

  def self.perform
    IPBlacklist::DelistVerifierService.call
  end

  # Schedule: ogni 1-6 ore (configurabile)
  def self.schedule
    Rails.application.config.ip_blacklist.delist.check_interval
  end

end
```

5.1 Obiettivi Decisional Logic

Il modulo decisionale deve:

1. **Aggregare** signals multipli in evidenze strutturate
2. **Calcolare** confidence score affidabile basato su segnali e contesto
3. **Raccomandare** transizioni di stato basate su soglie configurabili

4. **Prevenire** flapping e oscillazioni tra stati
5. **Gestire** casi edge e ambiguità

5.2 Time Windows per Aggregazione

Il sistema analizza i segnali su tre finestre temporali sovrapposte:

Window	Durata	Scopo	Azione Tipica
Short	5-15 min	Detection rapida di blacklist acute	ACTIVE → SUSPECT
Medium	1-6 ore	Analisi trend e pattern consolidati	SUSPECT → QUARANTINED
Long	24h-7 giorni	Valutazione reputazione globale	MONITORING → ACTIVE

Configurazione default:

```
windows:
  short: 15.minutes
  medium: 1.hour
  long: 24.hours
```

5.3 Confidence Score Calculation

Il confidence score rappresenta la probabilità che un IP sia effettivamente blacklistato, su scala 0.0-1.0.

5.3.1 Formula Base

```
confidence_score = weighted_avg(signals) * diversity_boost * recency_weight *
(1 - noise_penalty)
```

Componenti:

A) Weighted Average dei Signals

```
def calculate_base_confidence(signals)
  return 0.0 if signals.empty?

  total_weight = 0.0
  weighted_sum = 0.0

  signals.each do |signal|
    weight = signal_weight(signal)
    weighted_sum += signal.confidence * weight
    total_weight += weight
  end

  weighted_sum / total_weight
```

```
end

def signal_weight(signal)
  base_weight = case signal.signal_type
  when "hard_blacklist" then 1.0
  when "soft_blacklist" then 0.7
  when "reputation_issue" then 0.5
  when "unknown" then 0.3
  else 0.1
  end

  # Boost per pattern prioritari
  base_weight *= 1.2 if signal.pattern&.priority < 50

  base_weight
end
```

B) Provider Diversity Boost

```
def diversity_boost(signals)
  unique_providers = signals.map(&:provider_domain).uniq.count

  case unique_providers
  when 0..1 then 1.0      # nessun boost
  when 2 then 1.15        # 2 provider → +15%
  when 3 then 1.30        # 3 provider → +30%
  else 1.50               # 4+ provider → +50%
  end
end
```

C) Recency Weight (Time Decay)

```
def recency_weight(signals, window_start)
  # Segnali più recenti pesano di più
  now = Time.current
  window_duration = now - window_start

  weighted_sum = 0.0
  total_weight = 0.0

  signals.each do |signal|
    age = now - signal.detected_at
    # Decay esponenziale: peso = e^(-age/half_life)
    half_life = window_duration / 2
    recency = Math.exp(-age / half_life)

    weighted_sum += recency
    total_weight += 1
  end
```

```
  weighted_sum / total_weight
end
```

D) Noise Penalty

```
def noise_penalty(signals)
  # Penalizza se ci sono molti segnali "unknown" o bassa confidence
  low_confidence_count = signals.count { |s| s.confidence < 0.5 }
  unknown_count = signals.count { |s| s.signal_type == "unknown" }

  noise_ratio = (low_confidence_count + unknown_count).to_f / signals.size

  # Penalità max 20%
  [noise_ratio * 0.2, 0.2].min
end
```

5.3.2 Esempio Calcolo

Scenario: IP riceve 5 segnali in 15 minuti:

```
signals = [
  { type: "hard_blacklist", confidence: 0.95, provider: "gmail.com", age: 2.min
},
  { type: "hard_blacklist", confidence: 0.90, provider: "outlook.com", age:
5.min },
  { type: "hard_blacklist", confidence: 0.95, provider: "gmail.com", age: 8.min
},
  { type: "soft_blacklist", confidence: 0.70, provider: "yahoo.com", age:
10.min },
  { type: "reputation_issue", confidence: 0.60, provider: "gmail.com", age:
12.min }
]

# Step 1: Base confidence
weighted_avg = (0.95*1.0 + 0.90*1.0 + 0.95*1.0 + 0.70*0.7 + 0.60*0.5) /
(1.0+1.0+1.0+0.7+0.5)
             = 3.59 / 4.2 = 0.855

# Step 2: Diversity boost (3 provider diversi)
diversity = 1.30

# Step 3: Recency weight
# Segnali molto recenti (2-12 min su window 15 min)
recency = 0.95  # molto alto

# Step 4: Noise penalty
# 1 signal con confidence < 0.7 su 5 = 20%
noise = 0.20 * 0.2 = 0.04
```

```
# Final score
confidence = 0.855 * 1.30 * 0.95 * (1 - 0.04)
           = 1.070 * 0.96 = 1.027 → capped a 1.0

# Risultato: confidence = 1.0 (certezza assoluta)
```

5.4 Threshold Configuration

Le soglie definiscono quando un IP deve cambiare stato:

```
thresholds:
  # ACTIVE → MONITORING
  monitoring:
    min_signals: 1                    # almeno 1 segnale
    signal_types: ["hard_blacklist", "soft_blacklist", "reputation_issue"]
    window: short                     # 15 minuti
    confidence_min: 0.50              # confidence >= 50%
    cooldown: 5.minutes               # non più di 1 transizione ogni 5 min

  # MONITORING → SUSPECT
  suspect:
    min_signals: 2                    # almeno 2 segnali
    hard_signals: 1                   # di cui almeno 1 hard
    window: short                     # 15 minuti
    confidence_min: 0.70              # confidence >= 70%
    error_rate_spike: 0.30            # o 30% error rate vs baseline
    cooldown: 10.minutes

  # SUSPECT → QUARANTINED
  quarantine:
    min_signals: 3                    # almeno 3 segnali
    hard_signals: 2                   # di cui almeno 2 hard
    window: short                     # 15 minuti
    confidence_min: 0.80              # confidence >= 80%
    multiple_providers: 2             # almeno 2 provider diversi
    OR:
      hard_signals: 5                 # oppure 5 hard signal in medium window
      window: medium                  # 1 ora
    cooldown: 30.minutes

  # QUARANTINED → DELIST_CHECK
  delist_check:
    min_quarantine_duration: 24.hours  # durata minima quarantena

  # DELIST_CHECK → ACTIVE
  delist_success:
    probe_success: true               # almeno 1 probe successful
    no_new_signals: true              # nessun nuovo segnale in last 1h

  # Any State → MANUAL_REVIEW
```

```
  manual_review:
    consecutive_failures: 3           # 3 transizioni fallite consecutive
    inconclusive_probes: 5            # 5 probe inconclusivi
    operator_override: true           # flag manuale
```

5.5 State Transition Logic

5.5.1 MonitorService Implementation

```ruby
module IPBlacklist
  class MonitorService

    def self.call
      new.call
    end

    def call
      # Processa tutti gli IP con segnali recenti
      ips_with_signals = find_ips_with_recent_signals

      ips_with_signals.each do |ip_address|
        process_ip(ip_address)
      end
    end

    private

    def process_ip(ip_address)
      # 1. Raccogli signals per ogni window
      short_signals = collect_signals(ip_address, window: :short)
      medium_signals = collect_signals(ip_address, window: :medium)
      long_signals = collect_signals(ip_address, window: :long)

      # 2. Calcola confidence per ogni window
      short_confidence = calculate_confidence(short_signals, :short)
      medium_confidence = calculate_confidence(medium_signals, :medium)
      long_confidence = calculate_confidence(long_signals, :long)

      # 3. Crea evidence record
      evidence = create_evidence(ip_address, short_signals, short_confidence)

      # 4. Determina raccomandazione stato
      recommendation = determine_state_recommendation(
        ip_address,
        short_confidence,
        medium_confidence,
        long_confidence,
        short_signals,
        medium_signals
      )
```

```
      # 5. Esegui transizione se necessaria
      execute_transition(ip_address, recommendation, evidence) if
recommendation
    end

    def determine_state_recommendation(ip, short_conf, medium_conf, long_conf,
short_sig, medium_sig)
      current_state = ip.blacklist_status&.status || "active"
      thresholds = Rails.application.config.ip_blacklist.thresholds

      case current_state
      when "active"
        check_active_to_monitoring(short_sig, short_conf, thresholds)

      when "monitoring"
        check_monitoring_transitions(short_sig, medium_sig, short_conf,
thresholds)

      when "suspect"
        check_suspect_to_quarantine(short_sig, medium_sig, short_conf,
thresholds)

      when "quarantined"
        check_quarantine_to_delist(ip, thresholds)

      when "delist_check"
        nil  # gestito da DelistVerifierService

      when "manual_review"
        nil  # solo operatore può cambiare
      end
    end

    def check_active_to_monitoring(signals, confidence, thresholds)
      t = thresholds[:monitoring]

      return nil if signals.size < t[:min_signals]
      return nil if confidence < t[:confidence_min]
      return nil unless cooldown_elapsed?(ip_address, t[:cooldown])

      "monitoring"
    end

    def check_monitoring_transitions(short_sig, medium_sig, confidence,
thresholds)
      # Può andare a SUSPECT o tornare ad ACTIVE

      # Check SUSPECT
      t = thresholds[:suspect]
      hard_count = short_sig.count { |s| s.signal_type == "hard_blacklist" }

      if short_sig.size >= t[:min_signals] &&
         hard_count >= t[:hard_signals] &&
         confidence >= t[:confidence_min] &&
```

```
          cooldown_elapsed?(ip_address, t[:cooldown])
        return "suspect"
      end

      # Check ritorno ad ACTIVE (nessun segnale recente)
      if short_sig.empty? && medium_sig.size < 2
        return "active"
      end

      nil  # mantieni stato
    end

    def check_suspect_to_quarantine(short_sig, medium_sig, confidence,
thresholds)
      t = thresholds[:quarantine]

      hard_count = short_sig.count { |s| s.signal_type == "hard_blacklist" }
      provider_count = short_sig.map(&:provider_domain).uniq.size

      # Condizione primaria
      if short_sig.size >= t[:min_signals] &&
         hard_count >= t[:hard_signals] &&
         confidence >= t[:confidence_min] &&
         provider_count >= t[:multiple_providers] &&
         cooldown_elapsed?(ip_address, t[:cooldown])
        return "quarantined"
      end

      # Condizione alternativa (OR)
      hard_medium = medium_sig.count { |s| s.signal_type == "hard_blacklist" }
      if hard_medium >= t[:OR][:hard_signals]
        return "quarantined"
      end

      nil
    end

    def check_quarantine_to_delist(ip, thresholds)
      status = ip.blacklist_status
      t = thresholds[:delist_check]

      quarantine_duration = Time.current - status.status_changed_at

      if quarantine_duration >= t[:min_quarantine_duration]
        return "delist_check"
      end

      nil
    end

    def execute_transition(ip_address, new_state, evidence)
      IPBlacklist::QuarantineService.transition_to(
        ip_address: ip_address,
        new_state: new_state,
```

```
        evidence: evidence,
        actor: "System"
      )
    end

  end
end
```

5.5.2 Anti-Flapping Mechanisms

Problema: Evitare oscillazioni rapide tra stati (es. ACTIVE ↔ SUSPECT ogni 5 min)

Soluzioni implementate:

A) Cooldown Period

```
def cooldown_elapsed?(ip_address, cooldown_duration)
  last_transition = ip_address.blacklist_status&.status_changed_at
  return true if last_transition.nil?

  Time.current - last_transition >= cooldown_duration
end
```

B) Hysteresis (Soglie diverse per salita/discesa)

```
thresholds:
  suspect:
    confidence_enter: 0.70    # per entrare in SUSPECT
    confidence_exit: 0.50     # per uscire da SUSPECT (più basso)
```

C) Consecutive Signal Requirement

```
def requires_consecutive_detections?(transition)
  # Alcune transizioni richiedono conferma su 2 cicli consecutivi
  ["monitoring_to_suspect", "suspect_to_quarantine"].include?(transition)
end
```

D) Exponential Backoff per Re-Quarantine

```
def calculate_backoff_multiplier(ip_address)
  quarantine_count = ip_address.blacklist_status.quarantine_count
  base_multiplier = 1.5
```

```
  [base_multiplier ** quarantine_count, 10.0].min  # max 10x
end
```

5.6 Evidence Record Creation

Ogni ciclo di aggregazione crea un record `IPBlacklistEvidence`:

```
def create_evidence(ip_address, signals, confidence)
  IPBlacklistEvidence.create!(
    ip_address: ip_address,
    evidence_type: determine_evidence_type(signals),
    window_start: Time.current - config.windows.short,
    window_end: Time.current,
    signal_count: signals.size,
    aggregate_confidence: confidence,
    providers_affected: signals.map(&:provider_domain).uniq,
    patterns_matched: signals.map { |s| {
      id: s.pattern_id,
      name: s.pattern&.name,
      type: s.signal_type,
      confidence: s.confidence
    }},
    status_recommendation: recommendation,
    metadata: {
      hard_signal_count: signals.count { |s| s.signal_type == "hard_blacklist"
},
      soft_signal_count: signals.count { |s| s.signal_type == "soft_blacklist"
},
      provider_diversity: signals.map(&:provider_domain).uniq.size,
      rbl_names: signals.flat_map { |s| s.metadata["rbl_detected"]
}.compact.uniq,
      max_confidence: signals.map(&:confidence).max,
      avg_confidence: signals.map(&:confidence).sum / signals.size
    }
  )
end
```

5.7 Error Rate Spike Detection

Oltre ai pattern matching, il sistema può rilevare spike anomali nel error rate:

```
def detect_error_rate_spike(ip_address)
  # Baseline: error rate ultimi 7 giorni
  baseline_rate = calculate_baseline_error_rate(ip_address, 7.days)

  # Current: error rate ultima ora
  current_rate = calculate_current_error_rate(ip_address, 1.hour)

  # Spike se current > baseline * threshold
```

```
  spike_threshold = 2.0  # 2x il baseline

  if current_rate > baseline_rate * spike_threshold
    {
      spike_detected: true,
      baseline_rate: baseline_rate,
      current_rate: current_rate,
      multiplier: current_rate / baseline_rate
    }
  else
    { spike_detected: false }
  end
end

def calculate_error_rate(ip_address, period)
  # Query MessageDB deliveries
  total = ip_address.deliveries.where("timestamp > ?", Time.current -
period).count
  errors = ip_address.deliveries.where(
    "timestamp > ? AND status IN (?)",
    Time.current - period,
    ["HardFail", "SoftFail"]
  ).count

  return 0.0 if total == 0
  errors.to_f / total
end
```

4.1 Obiettivi Detection

Il modulo di detection deve:

1. **Catturare ogni potenziale segnale** di blacklist (zero falsi negativi)
2. **Classificare accuratamente** il tipo di problema (RBL, reputation, rate limit)
3. **Estrarre metadata** per analisi (provider, RBL name, severity)
4. **Gestire variabilità** linguistica e di formato dei messaggi SMTP
5. **Scalare** con volumi elevati senza impatto su delivery performance

4.2 Estrazione Dati da Delivery Records

Ogni `Delivery` record nel MessageDB contiene:

```
{
  status: "HardFail" | "SoftFail",
  code: 550,                          # SMTP status code
  output: "550-5.7.1 ...",            # Full SMTP response
  details: "...",                     # Postal-generated context
  ip_address_id: 42,                  # Which IP was used
  timestamp: 1737891234.567,
```

```
  # ... altri campi
}
```

Trigger di capture:

- `status IN ('HardFail', 'SoftFail')` → candidate per analisi
- `ip_address_id IS NOT NULL` → sappiamo quale IP ha causato l'errore
- `code >= 400` → errore SMTP (non success)

4.3 Pattern Matching Engine

4.3.1 Pattern Configuration Table

Table: `ip_blacklist_patterns`

```
{
  name: "Spamhaus ZEN RBL",
  pattern_type: "hard_blacklist",
  provider_filter: nil,               # match all providers
  code_regex: "^5[0-9]{2}$",          # 5xx codes
  message_regex: "\\b(zen\\.spamhaus\\.org|spamhaus)\\b",
  base_confidence: 0.95,
  enabled: true,
  priority: 10,                       # lower = higher priority
  metadata: {
    rbl_name: "zen.spamhaus.org",
    remediation_url: "https://www.spamhaus.org/lookup/",
    severity: "critical"
  }
}
```

4.3.2 Pattern Categories (Pre-configured)

A) Hard Blacklist - RBL Explicit Mentions

```
- name: "Spamhaus ZEN"
  code_regex: "^5(50|54|71)$"
  message_regex: "\\b(zen\\.spamhaus\\.org|spamhaus.*blocked)\\b"
  confidence: 0.95

- name: "Barracuda RBL"
  code_regex: "^5[0-9]{2}$"
  message_regex: "\\b(b\\.barracudacentral\\.org|barracuda.*block)\\b"
  confidence: 0.95

- name: "SpamCop"
  code_regex: "^5[0-9]{2}$"
  message_regex: "\\b(bl\\.spamcop\\.net|spamcop)\\b"
  confidence: 0.90
```

```
- name: "SORBS"
  code_regex: "^5[0-9]{2}$"
  message_regex: "\\b(dnsbl\\.sorbs\\.net|sorbs)\\b"
  confidence: 0.90

- name: "Generic RBL Mention"
  code_regex: "^5[0-9]{2}$"
  message_regex: "\\b(blacklist|blocklist|dnsbl|rbl)\\b"
  confidence: 0.85
```

B) IP-Specific Blocks (not domain/recipient)

```
- name: "IP Blacklisted"
  code_regex: "^5(50|54)$"
  message_regex: "\\bIP\\s*(address)?\\s*(is\\s*)?
(blocked|banned|blacklisted)\\b"
  confidence: 0.85

- name: "IP Reputation Low"
  code_regex: "^5[0-9]{2}$"
  message_regex: "\\b(IP|sender)\\s*reputation\\s*(too\\s*)?(low|poor|bad)\\b"
  confidence: 0.75

- name: "IP Not Allowed"
  code_regex: "^5[0-9]{2}$"
  message_regex: "\\bIP.*not\\s*allowed\\b"
  confidence: 0.70
```

C) Rate Limiting IP-Specific

```
- name: "IP Rate Limit Exceeded"
  code_regex: "^421$"
  message_regex:
"\\b(too\\s*many|rate\\s*limit).*\\b(from\\s*your\\s*IP|per\\s*IP)\\b"
  confidence: 0.70

- name: "IP Connection Limit"
  code_regex: "^421$"
  message_regex: "\\btoo\\s*many\\s*connections\\s*from\\s*(your\\s*)?IP\\b"
  confidence: 0.70
```

D) Reputation Issues

```
- name: "Spam Score High"
  code_regex: "^5(50|54)$"
  message_regex: "\\b(spam|spammy)\\s*(score|rating|content)\\b"
```

```
  confidence: 0.65

- name: "Policy Violation"
  code_regex: "^5[0-9]{2}$"
  message_regex: "\\bpolicy\\s*(violation|rejected)\\b"
  confidence: 0.60
```

E) Exclusion Patterns (NOT blacklist)

```
- name: "Recipient Not Found"
  code_regex: "^5[0-9]{2}$"
  message_regex: "\\b(user|recipient|mailbox)\\s*
(not\\s*found|unknown|does\\s*not\\s*exist)\\b"
  confidence: 0.0     # exclude from blacklist signals

- name: "Domain Not Exist"
  code_regex: "^5[0-9]{2}$"
  message_regex: "\\b(domain|host)\\s*not\\s*found\\b"
  confidence: 0.0

- name: "Quota Exceeded"
  code_regex: "^5[0-9]{2}$"
  message_regex: "\\b(quota|mailbox\\s*full)\\b"
  confidence: 0.0

- name: "Content Filter (not IP)"
  code_regex: "^5[0-9]{2}$"
  message_regex: "\\b(content|message)\\s*rejected\\b"
  confidence: 0.30    # low: potrebbe essere spam da questo IP
```

4.3.3 Multi-Language Support

Pattern regex devono considerare variazioni linguistiche comuni:

```
- name: "Blacklist (Multi-lang)"
  message_regex:
"\\b(blacklist|blocklist|lista\\s*negra|liste\\s*noire|schwarze\\s*liste)\\b"

- name: "Blocked (Multi-lang)"
  message_regex: "\\b(blocked|bloqué|bloccato|blockiert|geblokkeerd)\\b"
```

Strategia per lingue non coperte:

- Keyword detection su termini tecnici (RBL names, codici)
- Machine learning future: classificatore multi-lingua
- Fallback: signal type "UNKNOWN" con confidence basso

4.4 DetectorService Implementation

4.4.1 Service Interface

```
module IPBlacklist
  class DetectorService

    def self.call(delivery_id:, ip_address:, server:, smtp_code:,
smtp_message:, provider_domain:)
      new(
        delivery_id: delivery_id,
        ip_address: ip_address,
        server: server,
        smtp_code: smtp_code,
        smtp_message: smtp_message,
        provider_domain: provider_domain
      ).call
    end

    def call
      # 1. Carica pattern attivi
      patterns = load_active_patterns

      # 2. Applica pattern matching
      matched_patterns = match_patterns(patterns)

      # 3. Determina signal type e confidence
      signal_type, confidence, pattern = classify_signal(matched_patterns)

      # 4. Escludi se non rilevante
      return nil if confidence == 0.0 || signal_type.nil?

      # 5. Crea signal record
      create_signal(signal_type, confidence, pattern)

      # 6. Log per monitoring
      log_signal_detected
    end

    private

    def match_patterns(patterns)
      patterns.select do |pattern|
        code_matches?(pattern) && message_matches?(pattern)
      end.sort_by(&:priority)
    end

    def code_matches?(pattern)
      return true if pattern.code_regex.blank?
      Regexp.new(pattern.code_regex).match?(@smtp_code.to_s)
    end

    def message_matches?(pattern)
      regex = Regexp.new(pattern.message_regex, Regexp::IGNORECASE)
```

```
      regex.match?(@smtp_message)
    end

    def classify_signal(matched_patterns)
      return [nil, 0.0, nil] if matched_patterns.empty?

      # Prendi il pattern con priorità più alta (priority più basso)
      best_match = matched_patterns.first

      [
        best_match.pattern_type,
        best_match.base_confidence,
        best_match
      ]
    end

    def create_signal(signal_type, confidence, pattern)
      IPBlacklistSignal.create!(
        ip_address: @ip_address,
        server: @server,
        delivery_id: @delivery_id,
        signal_type: signal_type,
        provider_domain: @provider_domain,
        smtp_code: @smtp_code,
        smtp_enhanced_code: extract_enhanced_code,
        smtp_message: @smtp_message.truncate(500),
        pattern_id: pattern&.id,
        confidence: confidence,
        detected_at: Time.current,
        metadata: {
          pattern_name: pattern&.name,
          rbl_detected: extract_rbl_names
        }
      )
    end

    def extract_enhanced_code
      # Pattern: 5.7.1, 4.2.1, etc.
      @smtp_message.match(/(\d\.\d+\.\d+)/)[1]
    rescue
      nil
    end

    def extract_rbl_names
      # Estrai tutti i RBL names menzionati
      rbl_pattern = /\b([\w-]+\.
(?:spamhaus|barracuda|spamcop|sorbs|dnsbl)\.\w+)\b/
      @smtp_message.scan(rbl_pattern).flatten.uniq
    end

  end
end
```

4.4.2 Performance Considerations

Caching:

```
# Cache pattern attivi in Redis per 5 minuti
def load_active_patterns
  Rails.cache.fetch("ip_blacklist_patterns:active", expires_in: 5.minutes) do
    IPBlacklistPattern.where(enabled: true).order(:priority).to_a
  end
end
```

Async Processing (Opzionale):

```
# Se volume troppo alto, queue detection
class AsyncDetectorJob
  def perform(delivery_data)
    IPBlacklist::DetectorService.call(delivery_data)
  end
end
```

Timeout Protection:

```
def call
  Timeout.timeout(2.seconds) do
    # ... detection logic
  end
rescue Timeout::Error => e
  KLogger.error("Detection timeout for delivery #{@delivery_id}")
  nil  # fail-safe: non bloccare delivery pipeline
end
```

4.5 Signal Enrichment

Ogni signal viene arricchito con metadata per analisi successive:

```
signal.metadata = {
  pattern_name: "Spamhaus ZEN",
  pattern_priority: 10,
  rbl_detected: ["zen.spamhaus.org"],
  provider_domain: "gmail.com",
  provider_category: "major_freemail",  # gmail, outlook, yahoo
  smtp_enhanced_code: "5.7.1",
  detection_latency_ms: 45,
  full_smtp_response: @smtp_message,    # può essere troncato in DB
  server_id: @server.id,
```

```
  organization_id: @server.organization_id
}
```

4.6 False Positive Mitigation

Strategie per ridurre falsi positivi:

1. **Exclusion patterns**: Pattern che esplicitamente escludono errori non-IP
2. **Provider whitelist**: IP con reputazione verificata presso provider (FBL, SNDS)
3. **Context awareness**: Distinguere spike temporaneo da pattern persistente
4. **Confidence decay**: Segnali vecchi pesano meno nel calcolo aggregato
5. **Human feedback**: Operatori possono marcare segnali come "false positive"

```
# Feedback mechanism
class IPBlacklistSignal
  def mark_as_false_positive!(user, reason)
    update!(
      metadata: metadata.merge(
        false_positive: true,
        marked_by: user.id,
        fp_reason: reason,
        marked_at: Time.current
      )
    )

    # Ricalcola confidence per questo IP
    IPBlacklist::MonitorService.recalculate_for_ip(@ip_address)
  end
end
```

4.7 Monitoring Detection Quality

Metrics da trackare:

- Signal capture rate (signals/min)
- Pattern match distribution (quale pattern più frequente)
- Provider distribution (quale provider genera più segnali)
- False positive rate (dopo feedback umano)
- Detection latency (tempo capture → storage)

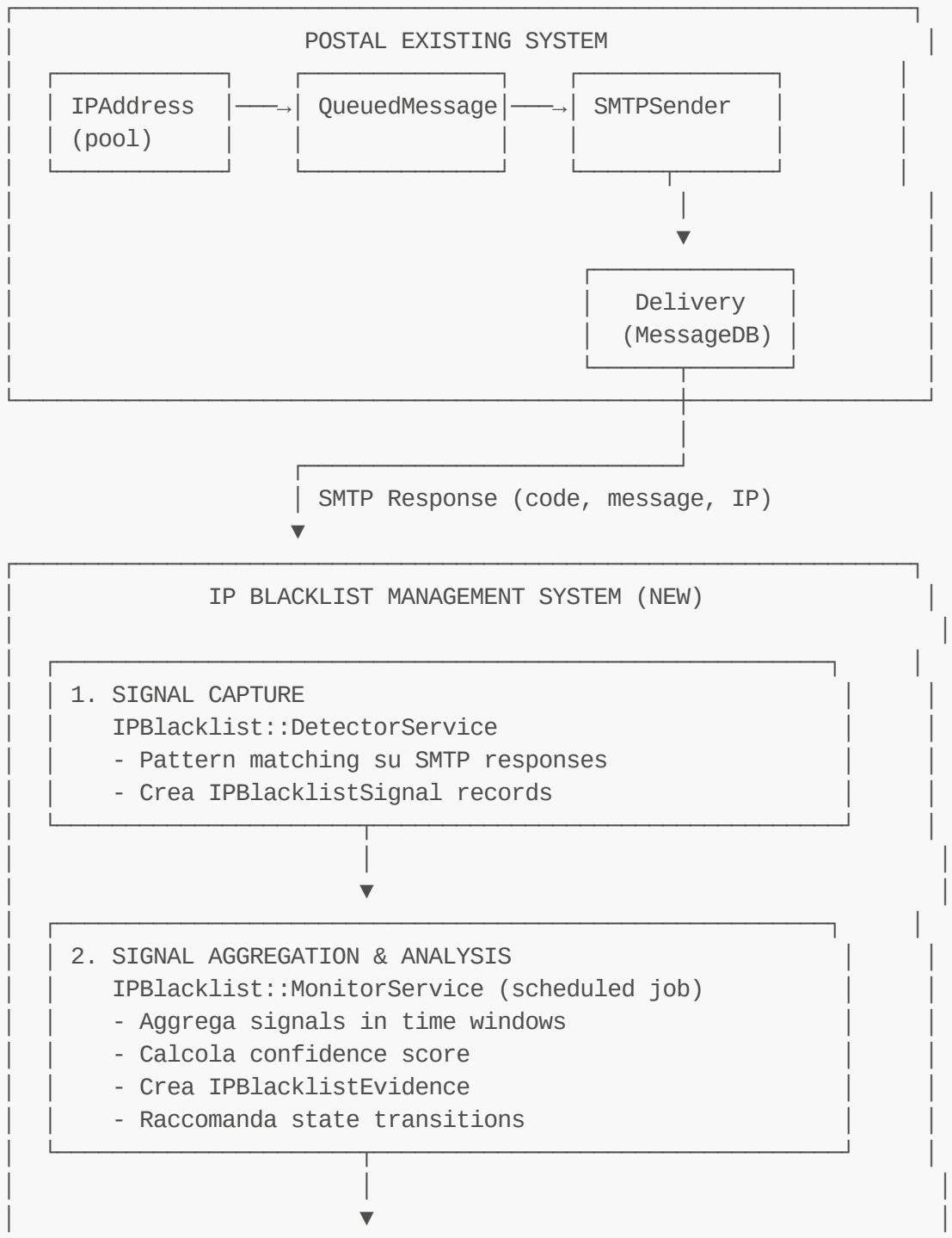
Dashboard query:

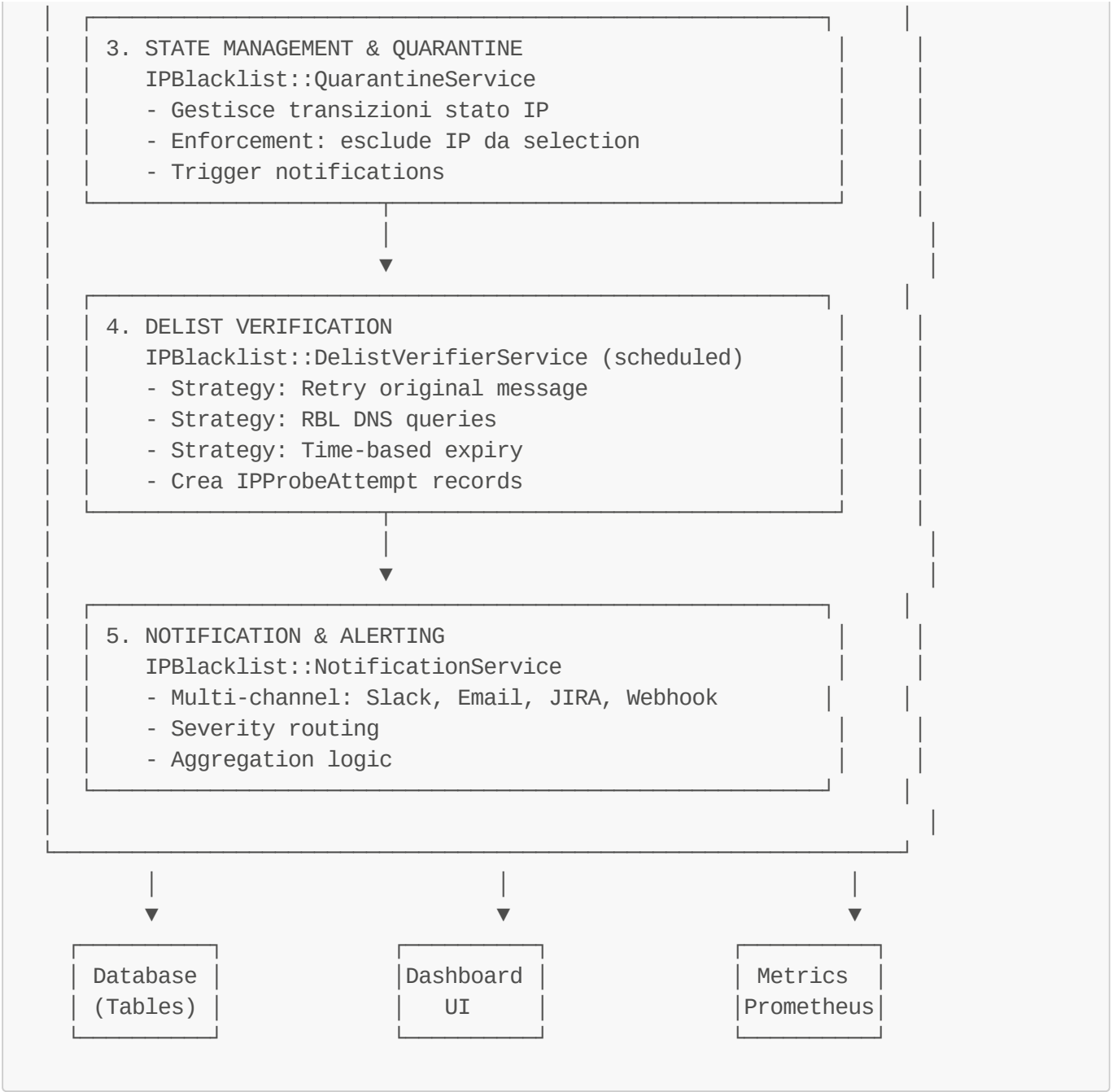
```
-- Top patterns matched last 24h
SELECT
  pattern_id,
  p.name,
  COUNT(*) as signal_count,
  AVG(confidence) as avg_confidence
```

```
FROM ip_blacklist_signals s
JOIN ip_blacklist_patterns p ON s.pattern_id = p.id
WHERE s.detected_at > NOW() - INTERVAL 24 HOUR
GROUP BY pattern_id
ORDER BY signal_count DESC
LIMIT 10;
```

3.1 Panoramica Componenti

Il sistema si integra nell'architettura esistente di Postal estendendo il modello IP Pool con componenti di monitoraggio, decisione e azione:





3.2 Database Schema Overview

Il sistema introduce 6 nuove tabelle nel database principale di Postal:

1. `ip_address_statuses`: Stato corrente e metadata di ogni IP
2. `ip_blacklist_signals`: Eventi grezzi catturati da SMTP responses (high-volume)
3. `ip_blacklist_patterns`: Pattern configurabili per detection (configuration)
4. `ip_blacklist_evidence`: Aggregazioni di signals per decisioni (analytical)
5. `ip_probe_attempts`: Log tentativi verifica delist (audit)
6. `ip_blacklist_audit_logs`: Audit trail completo di ogni cambio stato (compliance)

Relazioni principali:

- `IPAddress` (1) ↔ (1) `IPAddressStatus`
- `IPAddress` (1) ↔ (N) `IPBlacklistSignal`
- `IPAddress` (1) ↔ (N) `IPBlacklistEvidence`
- `IPAddress` (1) ↔ (N) `IPProbeAttempt`

- `IPAddress` (1) ↔ (N) `IPBlacklistAuditLog`

3.3 Integration Points con Postal Esistente

3.3.1 Hook in Delivery Flow

File: `app/lib/message_dequeuer/outgoing_message_processor.rb`

```
# Dopo creazione delivery
after_delivery :capture_blacklist_signal

def capture_blacklist_signal
  return unless @result.type.in?(["HardFail", "SoftFail"])
  return unless queued_message.ip_address.present?

  IPBlacklist::DetectorService.call(
    delivery_id: @delivery.id,
    ip_address: queued_message.ip_address,
    server: queued_message.server,
    smtp_code: @result.output.match(/^(\d{3})/)[1],
    smtp_message: @result.output,
    provider_domain: extract_domain(queued_message.message.rcpt_to)
  )
rescue => e
  # Fail-safe: errore detection non deve bloccare delivery
  KLogger.error("Blacklist detection failed: #{e.message}")
end
```

3.3.2 Modifica IP Selection Logic

File: `app/models/ip_address.rb`

```
# Override esistente
def self.select_by_priority
  # PRIMA: usable_for_sending.order("RAND() * priority DESC").first

  # DOPO: escludi IP non ACTIVE o MONITORING
  usable_for_blacklist_system
    .order("RAND() * priority DESC")
    .first
end

scope :usable_for_blacklist_system, -> {
  left_joins(:blacklist_status)
    .where(
      "ip_address_statuses.status IS NULL OR
       ip_address_statuses.status IN (?)",
      ["active", "monitoring"]
```

```
    )
}
```

3.3.3 Dashboard Integration

Nuove sezioni UI:

- `/ip_pools/:id/blacklist_status` - Overview stato IP del pool
- `/ip_addresses/:id/blacklist_history` - Timeline eventi specifico IP
- `/admin/blacklist_patterns` - Gestione pattern configurabili
- `/admin/blacklist_settings` - Configurazione soglie e thresholds

3.4 Background Jobs Architecture

Scheduler: Postal usa `app/lib/worker.rb` con job periodici

```
# app/lib/worker/jobs/ip_blacklist_monitor_job.rb
# Frequenza: ogni 5 minuti (configurabile)
# Funzione: Aggrega signals → evidence, trigger transizioni stato

# app/lib/worker/jobs/ip_delist_check_job.rb
# Frequenza: ogni 1 ora (configurabile)
# Funzione: Verifica IP in DELIST_CHECK, esegue probe

# app/lib/worker/jobs/ip_blacklist_cleanup_job.rb
# Frequenza: daily
# Funzione: Cleanup signals vecchi, archiviazione, pruning
```

3.5 Flusso End-to-End Esempio

Scenario: IP `203.0.113.42` viene blacklistato da Gmail

```
1. INVIO
   QueuedMessage allocato a IP 203.0.113.42
   ↓
   SMTPSender invia a gmail.com
   ↓
   Risposta: 550-5.7.1 blocked using zen.spamhaus.org

2. CAPTURE (immediate - in delivery callback)
   DetectorService analizza risposta
   ↓
   Pattern "Spamhaus RBL" matched (confidence 0.95)
   ↓
   Crea IPBlacklistSignal record

3. AGGREGATION (ogni 5 min - background job)
   MonitorService query signals recenti per IP 203.0.113.42
   ↓
```

```
   Trova 3 signals HARD_BLACKLIST in 15 min window
   ↓
   Calcola confidence score: 0.93
   ↓
   Crea IPBlacklistEvidence
   ↓
   Raccomanda: QUARANTINE

4. STATE TRANSITION (immediate dopo recommendation)
   QuarantineService
   ↓
   ACTIVE → QUARANTINED
   ↓
   Crea audit log
   ↓
   Trigger notification (severity: ERROR)

5. NOTIFICATION (immediate)
   NotificationService
   ↓
   POST to Slack webhook
   ↓
   Create JIRA ticket
   ↓
   Email a ops team

6. QUARANTINE PERIOD (24h configured)
   IP escluso da select_by_priority
   ↓
   Monitoring continua (altri signals? peggiora?)
   ↓
   Dopo 24h → DELIST_CHECK

7. VERIFICATION (dopo 24h - background job)
   DelistVerifierService
   ↓
   Strategy 1: Retry last failed message
   ↓
   Alloca IP 203.0.113.42 manualmente
   ↓
   Invia stesso messaggio a Gmail
   ↓
   Risultato: SUCCESS (250 OK)
   ↓
   Crea IPProbeAttempt (status: success)
   ↓
   DELIST_CHECK → ACTIVE
   ↓
   Notification: "IP recovered"

8. RECOVERY
   IP torna usable in pool
   ↓
   Monitoring rafforzato per 7 giorni
```

```
    ↓
    Segnali positivi → ritorno a normale
```

3.6 Scalabilità e Performance

Volume atteso:

- Signal capture: ~100-1000/min (dipende da volume invio)
- Aggregation job: processa batch 10K signals ogni 5 min
- State transitions: ~10-50/ora
- Delist checks: ~5-20/ora

Ottimizzazioni:

- Indici DB su `(ip_address_id, detected_at)` per query signals
- Partitioning/archiving signals dopo 90 giorni
- Caching pattern matching results (Redis)
- Async notifications (non bloccanti)
- Batch updates per state transitions

Resource requirements:

- DB storage: ~1GB/million signals (con retention 90d)
- Background worker: 1 thread dedicato
- Memory: ~200MB per worker process

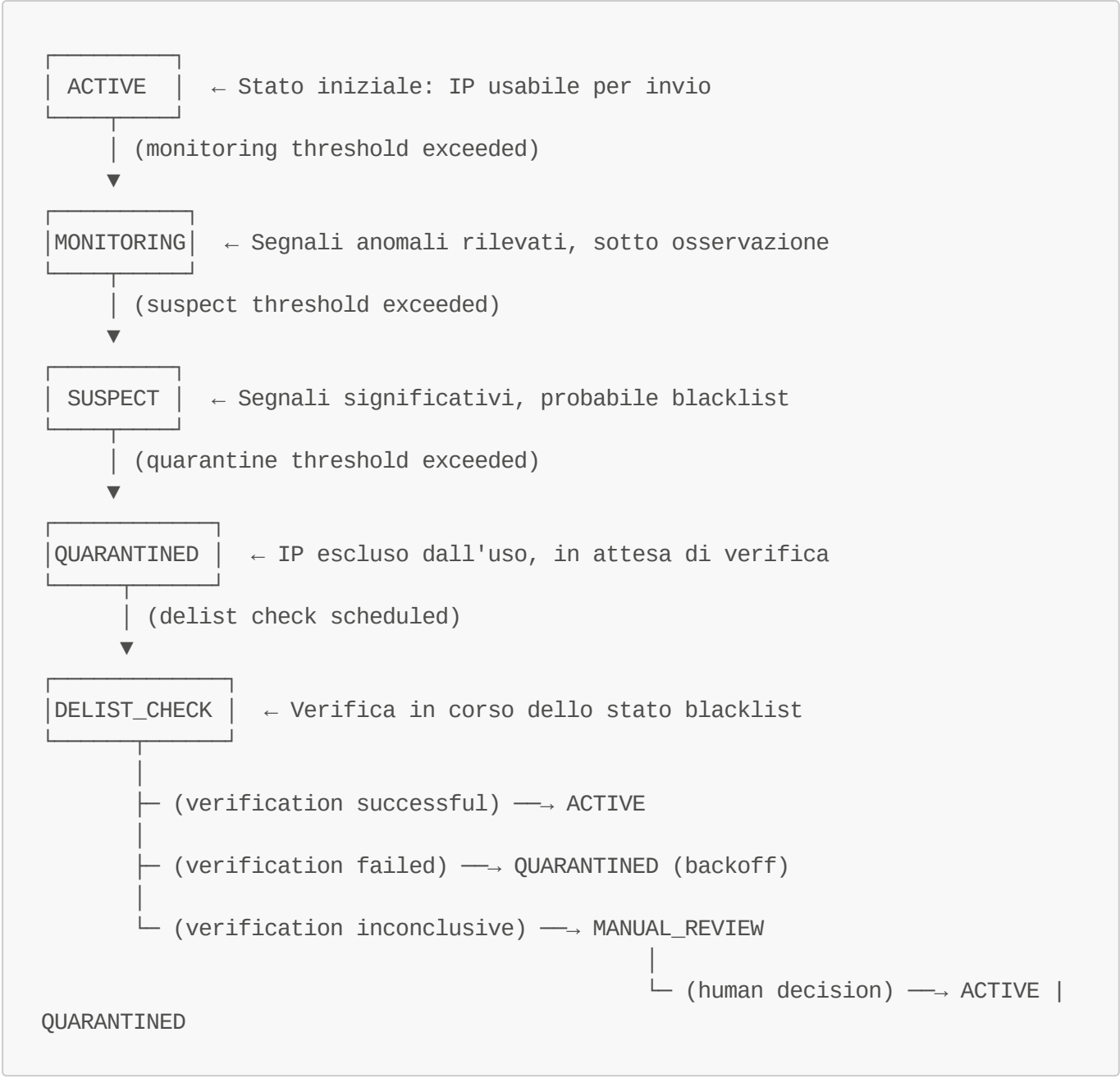
2.1 Terminologia

Termine	Definizione
Blacklist	Lista di IP address bloccati da un mail provider per invio spam, phishing o altre violazioni delle policy
RBL (Realtime Blackhole List)	Database DNS-based di IP blacklistati, consultabile pubblicamente (es. Spamhaus, SORBS)
Greylist	Tecnica temporanea di deferral delle email; non è una blacklist permanente
Reputation	Punteggio di affidabilità di un IP basato su comportamento storico di invio
Signal	Evento grezzo estratto da una risposta SMTP che indica possibile blacklist
Evidence	Aggregazione di più signal che supporta una decisione di cambio stato
Confidence Score	Valore 0-1 che rappresenta la certezza che un IP sia blacklistato
Quarantine	Stato di esclusione temporanea di un IP dal pool attivo
Delist	Processo di rimozione di un IP da una blacklist (naturale o tramite richiesta)
Probe	Tentativo controllato di verifica dello stato di un IP (invio test o query RBL)
Pool Health	Percentuale di IP attivi rispetto al totale del pool

Termine	Definizione
Flapping	Oscillazione rapida tra stati (es. ACTIVE ↔ QUARANTINED) da evitare

2.2 Stati del Ciclo di Vita IP

Un IP address può trovarsi in uno dei seguenti stati:



2.3 Definizione degli Stati

ACTIVE

- **Descrizione**: IP in pieno utilizzo, nessun segnale di problemi
- **Comportamento**: Selezionabile da `IPAddress.select_by_priority`
- **Transizione a MONITORING**: Rilevati segnali anomali (sotto soglia critica)

MONITORING

- **Descrizione**: IP ancora usabile ma sotto osservazione aumentata
- **Comportamento**: Selezionabile, ma con logging dettagliato e confidence tracking
- **Transizione a SUSPECT**: Accumulo di segnali blacklist (confidence > 0.70)
- **Transizione a ACTIVE**: Segnali rientrati, periodo di osservazione positivo

SUSPECT

- **Descrizione**: Forte sospetto di blacklist, uso limitato consigliato
- **Comportamento**: Selezionabile solo se nessun altro IP disponibile (degraded mode)
- **Transizione a QUARANTINED**: Soglia critica superata (confidence > 0.80)
- **Transizione a MONITORING**: Segnali diminuiti, possibile falso positivo

QUARANTINED

- **Descrizione**: IP confermato o altamente sospetto di blacklist, non utilizzabile
- **Comportamento**: **Escluso** da selezione automatica, nessun invio
- **Durata minima**: Configurabile (default: 24h)
- **Transizione a DELIST_CHECK**: Dopo durata minima, inizia verifica automatica
- **Transizione manuale a ACTIVE**: Solo tramite override operatore (logged)

DELIST_CHECK

- **Descrizione**: Verifica automatica in corso per determinare se IP è stato delistato
- **Comportamento**: Non utilizzabile, in attesa risultato probe/query
- **Transizione a ACTIVE**: Verifica positiva (IP delistato con successo)
- **Transizione a QUARANTINED**: Verifica negativa (IP ancora blacklistato) con backoff
- **Transizione a MANUAL_REVIEW**: Verifica inconclusiva o errori

MANUAL_REVIEW

- **Descrizione**: Sistema non può determinare stato, richiede intervento umano
- **Comportamento**: Non utilizzabile, alert critico generato
- **Transizione**: Solo manuale da operatore con motivazione obbligatoria

2.4 Tipi di Signal

Signal Type	Descrizione	Esempi SMTP	Confidence Base
HARD_BLACKLIST	Menzione esplicita di RBL o block permanente	`550 blocked using zen.spamhaus.org`	0.95
SOFT_BLACKLIST	Rate limiting o reputazione bassa IP-specific	`421 too many connections from your IP`	0.70
REPUTATION_ISSUE	Segnali di spam score o complaints	`550 sender reputation too low`	0.75
GREYLIST	Deferral temporaneo (non è blacklist)	`451 Greylisted, try again later`	0.10

Signal Type	Descrizione	Esempi SMTP	Confidence Base
UNKNOWN	Pattern non classificabile, da investigare	Messaggi ambigui	0.50

2.5 Metriche Chiave

- **Error Rate**: Percentuale di delivery failure (HardFail + SoftFail) su totale tentativi
- **Confidence Score**: Probabilità aggregata che un IP sia blacklistato (0.0 - 1.0)
- **Signal Density**: Numero di signal per unità di tempo (signal/min)
- **Provider Diversity**: Numero di provider distinti che segnalano problemi
- **Pool Health Ratio**: IP attivi / IP totali nel pool
- **Quarantine Duration**: Tempo medio di permanenza in quarantena
- **False Positive Rate**: IP quarantined erroneamente / totale quarantine
- **Delist Success Rate**: Verifiche delist riuscite / totale verifiche

---

1. Executive Summary
2. Glossario e Definizioni
3. Architettura Proposta
4. Detection Logic - Raccolta Segnali
5. Decisional Logic - Aggregazione e Scoring
6. Quarantine Strategy
7. Delist Verification Strategy
8. Notification System
9. Data Model
10. Implementation Components
11. Configuration System
12. Observability
13. Testing Strategy
14. Rollout Plan
15. Security & Access Control
16. Failure Modes & Recovery
17. Compliance & Retention
18. Open Questions & Future Enhancements

---

8. Notification System

8.1 Obiettivi Notification

1. **Multi-channel delivery**: Slack, Email, JIRA, Webhook generico
2. **Severity-based routing**: Eventi diversi → canali diversi
3. **Aggregation logic**: Evitare alert storm, raggruppare eventi simili
4. **Rich context**: Fornire tutte le info necessarie per azione immediata
5. **Reliable delivery**: Retry logic, fallback channels

8.2 Alert Levels e Routing

Severity	Events	Channels	Aggregation	SLA Response
INFO	IP → MONITORING, IP → ACTIVE	Log only, optional webhook	Daily digest	None
WARNING	IP → SUSPECT, Pool degraded	Slack, Email	Hourly digest	Review within 4h
ERROR	IP → QUARANTINED	Slack, Email, JIRA	Immediate + daily summary	Review within 1h
CRITICAL	Pool critical health, IP → MANUAL_REVIEW	Slack, Email, JIRA, PagerDuty	Immediate, no aggregation	Immediate action

8.3 Notification Payload Schema

```
{
  "alert_id": "uuid",
  "timestamp": "2026-01-26T10:30:00Z",
  "event_type": "ip_quarantined",
  "severity": "error",
  "source": "ip_blacklist_system",

  "subject": {
    "ip_address": "203.0.113.42",
    "ip_address_id": 42,
    "hostname": "mail-42.example.com",
    "pool_id": 5,
    "pool_name": "primary-pool"
  },

  "state_change": {
    "from": "suspect",
    "to": "quarantined",
    "trigger": "automatic",
    "actor": "System"
  },

  "evidence": {
    "confidence_score": 0.92,
    "signal_count": 5,
    "window": "last 15 minutes",
    "providers_affected": ["gmail.com", "outlook.com"],
    "patterns_matched": [
      {
        "name": "Spamhaus ZEN RBL",
        "type": "hard_blacklist",
        "confidence": 0.95,
        "occurrences": 3
      },
      {
```

```
        "name": "IP Reputation Low",
        "type": "reputation_issue",
        "confidence": 0.75,
        "occurrences": 2
      }
    ],
    "rbl_detected": ["zen.spamhaus.org"],
    "sample_errors": [
      {
        "timestamp": "2026-01-26T10:25:00Z",
        "provider": "gmail.com",
        "smtp_code": 550,
        "message": "550-5.7.1 ... blocked using zen.spamhaus.org"
      }
    ]
  },

  "impact": {
    "pool_health": "degraded",
    "active_ips_remaining": 4,
    "total_ips": 5,
    "estimated_capacity_loss": "20%",
    "quarantine_duration_estimated": "48 hours"
  },

  "suggested_actions": [
    "Check Spamhaus listing: https://www.spamhaus.org/lookup/?ip=203.0.113.42",
    "Review recent campaigns sent from this IP",
    "Verify no compromised accounts",
    "Check server logs for suspicious activity"
  ],

  "links": {
    "dashboard": "https://postal.example.com/ip_pools/5/blacklist_status",
    "ip_detail": "https://postal.example.com/ip_addresses/42",
    "audit_log": "https://postal.example.com/ip_addresses/42/blacklist_log",
    "evidence": "https://postal.example.com/ip_blacklist_evidence/123"
  },

  "metadata": {
    "server_ids": [1, 2, 3],
    "organization_id": 10,
    "notification_id": "notif-uuid"
  }
}
```

8.4 NotificationService Implementation

```
module IPBlacklist
  class NotificationService
```

```ruby
    def self.call(event:, severity:, data:)
      new(event, severity, data).deliver
    end

    def deliver
      # 1. Build payload
      payload = build_payload

      # 2. Determine channels based on severity
      channels = determine_channels

      # 3. Check aggregation
      if should_aggregate?
        queue_for_aggregation(payload, channels)
      else
        send_immediately(payload, channels)
      end
    end

    private

    def build_payload
      {
        alert_id: SecureRandom.uuid,
        timestamp: Time.current.iso8601,
        event_type: @event,
        severity: @severity,
        source: "ip_blacklist_system",
        subject: build_subject,
        state_change: @data[:state_change],
        evidence: @data[:evidence],
        impact: @data[:impact],
        suggested_actions: generate_suggested_actions,
        links: generate_links,
        metadata: @data[:metadata]
      }
    end

    def determine_channels
      config = Rails.application.config.ip_blacklist.notifications

      config.channels.select do |channel|
        # Filter by events
        channel[:events] == "all" || channel[:events].include?(@event)
      end.select do |channel|
        # Filter by severity
        channel_severity = channel_severity_level(channel)
        current_severity = severity_level(@severity)

        current_severity >= channel_severity
      end
    end

    def severity_level(severity)
```

```
    { "info" => 0, "warning" => 1, "error" => 2, "critical" => 3 }[severity]
  end

  def should_aggregate?
    return false if @severity == "critical"

    config = Rails.application.config.ip_blacklist.notifications
    config.aggregate_warnings && @severity.in?(["info", "warning"])
  end

  def send_immediately(payload, channels)
    channels.each do |channel|
      send_to_channel(channel, payload)
    end
  end

  def send_to_channel(channel, payload)
    case channel[:type]
    when "slack"
      send_to_slack(channel, payload)
    when "email"
      send_to_email(channel, payload)
    when "jira"
      create_jira_ticket(channel, payload)
    when "webhook"
      send_to_webhook(channel, payload)
    end
  rescue => e
    KLogger.error("Notification delivery failed", {
      channel: channel[:type],
      error: e.message
    })

    # Fallback: store in DB for manual review
    store_failed_notification(channel, payload, e)
  end

  def send_to_slack(channel, payload)
    formatted = format_for_slack(payload)

    response = HTTP.post(
      channel[:webhook_url],
      json: formatted,
      timeout: 5
    )

    unless response.status.success?
      raise "Slack API error: #{response.code}"
    end

    log_notification_sent("slack", payload[:alert_id])
  end

  def format_for_slack(payload)
```

```
      {
        text: "IP Blacklist Alert: #{payload[:event_type]}",
        attachments: [
          {
            color: severity_color(payload[:severity]),
            title: "IP #{payload[:subject][:ip_address]} → #
{payload[:state_change][:to].upcase}",
            fields: [
              {
                title: "Pool",
                value: payload[:subject][:pool_name],
                short: true
              },
              {
                title: "Confidence",
                value: "#{(payload[:evidence][:confidence_score] *
100).round}%",
                short: true
              },
              {
                title: "Providers Affected",
                value: payload[:evidence][:providers_affected].join(", "),
                short: false
              },
              {
                title: "Pool Health",
                value: "#{payload[:impact][:active_ips_remaining]}/#
{payload[:impact][:total_ips]} IPs active",
                short: true
              }
            ],
            actions: [
              {
                type: "button",
                text: "View Dashboard",
                url: payload[:links][:dashboard]
              },
              {
                type: "button",
                text: "View Details",
                url: payload[:links][:ip_detail]
              }
            ],
            footer: "Postal IP Blacklist Monitor",
            ts: Time.current.to_i
          }
        ]
      }
    end

    def send_to_email(channel, payload)
      IPBlacklistMailer.alert_notification(
        recipients: channel[:recipients],
        payload: payload
```

```
      ).deliver_later
    end

    def create_jira_ticket(channel, payload)
      jira_client = JIRA::Client.new(
        username: channel[:username],
        password: channel[:api_token],
        site: channel[:site],
        context_path: '',
        auth_type: :basic
      )

      issue = jira_client.Issue.build
      issue.save({
        "fields" => {
          "project" => { "key" => channel[:project] },
          "summary" => "IP #{payload[:subject][:ip_address]} quarantined - #
{payload[:evidence][:providers_affected].join(', ')}",
          "description" => format_for_jira(payload),
          "issuetype" => { "name" => channel[:issue_type] || "Incident" },
          "priority" => { "name" => jira_priority(payload[:severity]) },
          "labels" => ["ip_blacklist", "automated", payload[:subject]
[:pool_name]]
        }
      })

      log_notification_sent("jira", payload[:alert_id], issue.key)
    end

    def send_to_webhook(channel, payload)
      response = HTTP.timeout(10).post(
        channel[:url],
        json: payload,
        headers: {
          "Content-Type" => "application/json",
          "X-Postal-Event" => payload[:event_type],
          "X-Postal-Severity" => payload[:severity]
        }
      )

      unless response.status.success?
        raise "Webhook error: #{response.code}"
      end

      log_notification_sent("webhook", payload[:alert_id])
    end

    def generate_suggested_actions
      actions = []

      # RBL-specific actions
      if @data[:evidence]&.dig(:rbl_detected)&.any?
        rbls = @data[:evidence][:rbl_detected]
        rbls.each do |rbl|
```

```
          actions << "Check #{rbl} listing: #{rbl_lookup_url(rbl,
@data[:ip_address])}"
        end
      end

      # Generic actions
      actions << "Review recent email campaigns from this IP"
      actions << "Check server logs for suspicious activity"
      actions << "Verify no compromised accounts sending spam"

      # Pool health specific
      if @data[:impact]&.dig(:pool_health) == "critical"
        actions << "URGENT: Pool capacity critical, consider manual override or
alternative pool"
      end

      actions
    end

    def rbl_lookup_url(rbl, ip)
      case rbl
      when /spamhaus/
        "https://www.spamhaus.org/lookup/?ip=#{ip}"
      when /barracuda/
        "https://www.barracudacentral.org/lookups/lookup-reputation?ip=#{ip}"
      when /spamcop/
        "https://www.spamcop.net/bl.shtml?ip=#{ip}"
      else
        "https://mxtoolbox.com/SuperTool.aspx?action=blacklist%3a#{ip}"
      end
    end

  end
end
```

8.5 Aggregation Strategy

Per eventi INFO e WARNING, aggregare in digest periodici:

```
class IPBlacklistNotificationAggregator

  def self.send_digest(period: :hourly)
    new(period).send_digest
  end

  def send_digest
    # Raccogli notifiche aggregate dal periodo
    notifications = collect_pending_notifications

    return if notifications.empty?
```

```
    # Group by pool and severity
    grouped = notifications.group_by { |n| [n.pool_id, n.severity] }

    # Send one digest per group
    grouped.each do |(pool_id, severity), notifs|
      send_digest_for_group(pool_id, severity, notifs)
    end

    # Mark as sent
    mark_notifications_sent(notifications)
  end

  private

  def send_digest_for_group(pool_id, severity, notifications)
    pool = IPPool.find(pool_id)

    payload = {
      alert_id: SecureRandom.uuid,
      timestamp: Time.current.iso8601,
      event_type: "digest",
      severity: severity,
      period: @period,
      pool: {
        id: pool.id,
        name: pool.name
      },
      summary: {
        total_events: notifications.size,
        ips_affected: notifications.map(&:ip_address_id).uniq.size,
        event_breakdown:
notifications.group_by(&:event_type).transform_values(&:count)
      },
      events: notifications.map { |n| summarize_event(n) }
    }

    # Send via configured channels
    IPBlacklist::NotificationService.send_digest(payload)
  end

end
```

---

9. Data Model

9.1 Database Tables Schema

Le seguenti tabelle vengono aggiunte al database principale di Postal.

9.1.1 ip_address_statuses

Stato corrente di ogni IP address.

```ruby
# db/migrate/YYYYMMDDHHMMSS_create_ip_address_statuses.rb
class CreateIPAddressStatuses < ActiveRecord::Migration[7.1]

  def change
    create_table :ip_address_statuses do |t|
      t.references :ip_address, foreign_key: true, null: false, index: {
unique: true }
      t.string :status, null: false, limit: 50
      t.datetime :status_changed_at, null: false, precision: nil
      t.integer :quarantine_count, default: 0, null: false
      t.datetime :last_quarantined_at, precision: nil
      t.decimal :confidence_score, precision: 5, scale: 4, default: 0.0
      t.jsonb :metadata, default: {}
      t.timestamps precision: nil

      t.index :status
      t.index :status_changed_at
      t.index [:status, :status_changed_at]
    end
  end

end
```

Fields:

- `ip_address_id`: FK to ip_addresses
- `status`: active, monitoring, suspect, quarantined, delist_check, manual_review
- `status_changed_at`: Timestamp ultima transizione
- `quarantine_count`: Numero volte IP è stato quarantined (per backoff)
- `last_quarantined_at`: Timestamp ultima quarantena
- `confidence_score`: Score aggregato corrente (0.0-1.0)
- `metadata`: JSON con info aggiuntive (duration, evidence_id, etc.)

9.1.2 ip_blacklist_signals

Eventi grezzi catturati dalle risposte SMTP (high-volume).

```ruby
# db/migrate/YYYYMMDDHHMMSS_create_ip_blacklist_signals.rb
class CreateIPBlacklistSignals < ActiveRecord::Migration[7.1]

  def change
    create_table :ip_blacklist_signals do |t|
      t.references :ip_address, foreign_key: true, null: false
      t.references :server, foreign_key: true, null: false
      t.bigint :delivery_id  # MessageDB delivery ID
      t.string :signal_type, null: false, limit: 50
      t.string :provider_domain, limit: 255
      t.integer :smtp_code
      t.string :smtp_enhanced_code, limit: 20
```

```
      t.text :smtp_message, limit: 1000
      t.integer :pattern_id
      t.decimal :confidence, precision: 5, scale: 4
      t.datetime :detected_at, null: false, precision: nil
      t.jsonb :metadata, default: {}

      t.index [:ip_address_id, :detected_at], name:
"index_signals_on_ip_and_time"
      t.index [:signal_type, :detected_at]
      t.index :provider_domain
      t.index :detected_at
      t.index :pattern_id
    end
  end

end
```

Retention Policy:

- Mantenere 90 giorni (configurabile)
- Scheduled cleanup job per archiving

9.1.3 ip_blacklist_patterns

Pattern configurabili per detection (configuration table).

```
# db/migrate/YYYYMMDDHHMMSS_create_ip_blacklist_patterns.rb
class CreateIPBlacklistPatterns < ActiveRecord::Migration[7.1]

  def change
    create_table :ip_blacklist_patterns do |t|
      t.string :name, null: false, limit: 255
      t.string :pattern_type, null: false, limit: 50
      t.string :provider_filter, limit: 255
      t.string :code_regex, limit: 500
      t.text :message_regex, null: false
      t.decimal :base_confidence, precision: 5, scale: 4, default: 0.8
      t.boolean :enabled, default: true, null: false
      t.integer :priority, default: 100, null: false
      t.jsonb :metadata, default: {}
      t.timestamps precision: nil

      t.index [:enabled, :priority]
      t.index :pattern_type
    end
  end

end
```

Seeds: Popolare con pattern predefiniti (vedi sezione 4.3.2)

9.1.4 ip_blacklist_evidence

Aggregazioni di signals per decision-making.

```
# db/migrate/YYYYMMDDHHMMSS_create_ip_blacklist_evidence.rb
class CreateIPBlacklistEvidence < ActiveRecord::Migration[7.1]

  def change
    create_table :ip_blacklist_evidence do |t|
      t.references :ip_address, foreign_key: true, null: false
      t.string :evidence_type, null: false, limit: 50
      t.datetime :window_start, null: false, precision: nil
      t.datetime :window_end, null: false, precision: nil
      t.integer :signal_count, default: 0, null: false
      t.decimal :aggregate_confidence, precision: 5, scale: 4
      t.jsonb :providers_affected, default: []
      t.jsonb :patterns_matched, default: []
      t.string :status_recommendation, limit: 50
      t.jsonb :metadata, default: {}
      t.timestamps precision: nil

      t.index [:ip_address_id, :window_start], name:
"index_evidence_on_ip_and_window"
      t.index :status_recommendation
      t.index :window_end
    end
  end

end
```

9.1.5 ip_probe_attempts

Log di ogni tentativo di verifica delist.

```
# db/migrate/YYYYMMDDHHMMSS_create_ip_probe_attempts.rb
class CreateIPProbeAttempts < ActiveRecord::Migration[7.1]

  def change
    create_table :ip_probe_attempts do |t|
      t.references :ip_address, foreign_key: true, null: false
      t.string :probe_type, null: false, limit: 50
      t.string :status, null: false, limit: 50
      t.text :result_details
      t.datetime :attempted_at, null: false, precision: nil
      t.datetime :completed_at, precision: nil
      t.jsonb :metadata, default: {}

      t.index [:ip_address_id, :attempted_at], name:
"index_probes_on_ip_and_time"
      t.index :status
```

```
      t.index :probe_type
    end
  end

end
```

9.1.6 ip_blacklist_audit_logs

Audit trail completo per compliance.

```
# db/migrate/YYYYMMDDHHMMSS_create_ip_blacklist_audit_logs.rb
class CreateIPBlacklistAuditLogs < ActiveRecord::Migration[7.1]

  def change
    create_table :ip_blacklist_audit_logs do |t|
      t.references :ip_address, foreign_key: true, null: false
      t.string :event_type, null: false, limit: 50
      t.string :from_status, limit: 50
      t.string :to_status, limit: 50
      t.integer :actor_id
      t.string :actor_type, limit: 50
      t.text :reason
      t.jsonb :evidence_snapshot, default: {}
      t.datetime :occurred_at, null: false, precision: nil

      t.index [:ip_address_id, :occurred_at], name:
"index_audit_on_ip_and_time"
      t.index :event_type
      t.index [:actor_id, :actor_type]
      t.index :occurred_at
    end
  end

end
```

9.2 ActiveRecord Models

9.2.1 IPAddressStatus

```
# app/models/ip_address_status.rb
# frozen_string_literal: true

class IPAddressStatus < ApplicationRecord

  STATES = %w[
    active
    monitoring
    suspect
    quarantined
```

```
    delist_check
    manual_review
  ].freeze

  USABLE_STATES = %w[active monitoring].freeze

  belongs_to :ip_address

  validates :status, inclusion: { in: STATES }
  validates :ip_address_id, uniqueness: true

  scope :active, -> { where(status: "active") }
  scope :quarantined, -> { where(status: "quarantined") }
  scope :usable, -> { where(status: USABLE_STATES) }

  def quarantined?
    status == "quarantined"
  end

  def usable?
    status.in?(USABLE_STATES)
  end

  def requires_manual_review?
    status == "manual_review"
  end

end
```

9.2.2 IPBlacklistSignal

```
# app/models/ip_blacklist_signal.rb
# frozen_string_literal: true

class IPBlacklistSignal < ApplicationRecord

  SIGNAL_TYPES = %w[
    hard_blacklist
    soft_blacklist
    reputation_issue
    greylist
    unknown
  ].freeze

  belongs_to :ip_address
  belongs_to :server
  belongs_to :pattern, class_name: "IPBlacklistPattern", optional: true

  validates :signal_type, inclusion: { in: SIGNAL_TYPES }

  scope :recent, ->(duration = 1.hour) { where("detected_at > ?", duration.ago)
```

```
}
  scope :by_type, ->(type) { where(signal_type: type) }
  scope :high_confidence, -> { where("confidence >= ?", 0.75) }

  def false_positive?
    metadata["false_positive"] == true
  end

  def mark_as_false_positive!(user, reason)
    update!(
      metadata: metadata.merge(
        false_positive: true,
        marked_by: user.id,
        fp_reason: reason,
        marked_at: Time.current.to_s
      )
    )
  end

end
```

9.2.3 IPBlacklistPattern

```
# app/models/ip_blacklist_pattern.rb
# frozen_string_literal: true

class IPBlacklistPattern < ApplicationRecord

  PATTERN_TYPES = %w[
    hard_blacklist
    soft_blacklist
    reputation_issue
    greylist
    exclusion
  ].freeze

  has_many :signals, class_name: "IPBlacklistSignal", foreign_key: :pattern_id

  validates :name, presence: true
  validates :pattern_type, inclusion: { in: PATTERN_TYPES }
  validates :message_regex, presence: true

  scope :enabled, -> { where(enabled: true) }
  scope :ordered, -> { order(:priority, :name) }

  def matches_code?(code)
    return true if code_regex.blank?
    Regexp.new(code_regex).match?(code.to_s)
  end

  def matches_message?(message)
```

```
      regex = Regexp.new(message_regex, Regexp::IGNORECASE)
      regex.match?(message)
    end

    def matches?(code, message)
      matches_code?(code) && matches_message?(message)
    end

  end
```

9.2.4 IPAddress Extensions

```
# app/models/ip_address.rb (additions)
class IPAddress < ApplicationRecord

  has_one :blacklist_status, class_name: "IPAddressStatus", dependent: :destroy
  has_many :blacklist_signals, class_name: "IPBlacklistSignal", dependent:
:destroy
  has_many :blacklist_evidence, class_name: "IPBlacklistEvidence", dependent:
:destroy
  has_many :probe_attempts, class_name: "IPProbeAttempt", dependent: :destroy
  has_many :blacklist_audit_logs, class_name: "IPBlacklistAuditLog", dependent:
:destroy

  scope :usable_for_sending, -> {
    left_joins(:blacklist_status)
      .where(
        "ip_address_statuses.id IS NULL OR
         ip_address_statuses.status IN (?)",
        IPAddressStatus::USABLE_STATES
      )
  }

  def current_blacklist_status
    blacklist_status&.status || "active"
  end

  def quarantined?
    blacklist_status&.quarantined? || false
  end

  def blacklist_confidence
    blacklist_status&.confidence_score || 0.0
  end

end
```

9.3 Data Retention Policy

```
retention:
  ip_blacklist_signals: 90.days
  ip_blacklist_evidence: 180.days
  ip_probe_attempts: 90.days
  ip_blacklist_audit_logs: 2.years  # compliance requirement
```

Cleanup Job:

```
class IPBlacklistCleanupJob

  def perform
    cleanup_signals
    cleanup_evidence
    cleanup_probes
    # audit logs kept indefinitely
  end

  private

  def cleanup_signals
    cutoff = config.retention.ip_blacklist_signals.ago
    deleted = IPBlacklistSignal.where("detected_at < ?", cutoff).delete_all
    KLogger.info("Cleaned up #{deleted} old blacklist signals")
  end

end
```

---

10. Implementation Components

10.1 Service Objects Overview

Tutti i service objects seguono il pattern:

```
module IPBlacklist
  class ServiceName

    def self.call(*args)
      new(*args).call
    end

    def call
      # implementation
    end

  end
end
```

Services da implementare:

1. **DetectorService** (sezione 4.4)
   - Analizza delivery records
   - Applica pattern matching
   - Crea signal records
2. **MonitorService** (sezione 5.5)
   - Aggrega signals
   - Calcola confidence
   - Raccomanda transizioni
3. **QuarantineService** (sezione 6.4)
   - Gestisce state transitions
   - Enforcement
   - Trigger notifications
4. **DelistVerifierService** (sezione 7.6)
   - Coordina strategie delist
   - Gestisce probe attempts
   - Aggiorna stati
5. **NotificationService** (sezione 8.4)
   - Multi-channel delivery
   - Formatting
   - Retry logic

10.2 Background Jobs

```
# app/lib/worker/jobs/ip_blacklist_monitor_job.rb
module Worker
  module Jobs
    class IPBlacklistMonitorJob

      def self.perform
        IPBlacklist::MonitorService.call
      end

      def self.schedule
        "*/5 * * * *"  # ogni 5 minuti
      end

    end
  end
end

# app/lib/worker/jobs/ip_delist_check_job.rb
```

```
module Worker
  module Jobs
    class IPDelistCheckJob

      def self.perform
        IPBlacklist::DelistVerifierService.call
      end

      def self.schedule
        "0 * * * *"  # ogni ora
      end

    end
  end
end

# app/lib/worker/jobs/ip_blacklist_cleanup_job.rb
module Worker
  module Jobs
    class IPBlacklistCleanupJob

      def self.perform
        IPBlacklistCleanupService.call
      end

      def self.schedule
        "0 2 * * *"  # daily at 2 AM
      end

    end
  end
end
```

10.3 Controllers

10.3.1 IP Pool Blacklist Status Controller

```
# app/controllers/ip_pool_blacklist_controller.rb
class IPPoolBlacklistController < ApplicationController

  before_action :find_ip_pool

  def status
    @metrics = @ip_pool.health_metrics
    @ip_addresses = @ip_pool.ip_addresses
                            .includes(:blacklist_status)
                            .order("ip_address_statuses.status DESC")
  end

  def recent_signals
    @signals = IPBlacklistSignal
```

```
                .joins(ip_address: :ip_pool)
                .where(ip_addresses: { ip_pool_id: @ip_pool.id })
                .recent(24.hours)
                .includes(:ip_address, :pattern)
                .order(detected_at: :desc)
                .limit(100)
  end

  private

  def find_ip_pool
    @ip_pool = IPPool.find(params[:ip_pool_id])
  end

end
```

10.3.2 IP Address Blacklist Controller

```
# app/controllers/ip_address_blacklist_controller.rb
class IPAddressBlacklistController < ApplicationController

  before_action :find_ip_address

  def show
    @status = @ip_address.blacklist_status
    @recent_signals = @ip_address.blacklist_signals.recent(7.days).limit(50)
    @recent_evidence = @ip_address.blacklist_evidence.order(window_end:
:desc).limit(10)
    @probe_attempts = @ip_address.probe_attempts.order(attempted_at:
:desc).limit(10)
  end

  def history
    @audit_logs = @ip_address.blacklist_audit_logs
                             .order(occurred_at: :desc)
                             .page(params[:page])
  end

  def force_state
    authorize @ip_address, :force_quarantine?

    result = IPBlacklist::QuarantineService.transition_to(
      ip_address: @ip_address,
      new_state: params[:new_state],
      evidence: nil,
      actor: current_user,
      reason: params[:reason]
    )

    if result
      flash[:success] = "IP state changed to #{params[:new_state]}"
```

```
      else
        flash[:error] = "Failed to change state"
      end

      redirect_to ip_address_blacklist_path(@ip_address)
    end

    private

    def find_ip_address
      @ip_address = IPAddress.find(params[:ip_address_id])
    end

  end
```

10.4 Mailers

```
# app/mailers/ip_blacklist_mailer.rb
class IPBlacklistMailer < ApplicationMailer

  def alert_notification(recipients:, payload:)
    @payload = payload

    mail(
      to: recipients,
      subject: "IP Blacklist Alert: #{payload[:subject][:ip_address]} - #
{payload[:event_type]}",
      template_name: "alert_notification"
    )
  end

  def digest(recipients:, payload:)
    @payload = payload

    mail(
      to: recipients,
      subject: "IP Blacklist Daily Digest - #{payload[:pool][:name]}",
      template_name: "digest"
    )
  end

end
```

---

11. Configuration System

11.1 Configuration File Structure

File: `config/postal/ip_blacklist.yml`

```yaml
ip_blacklist:
  # Global enable/disable
  enabled: true

  # Monitoring windows
  monitoring:
    signal_capture: true
    short_window: 15.minutes
    medium_window: 1.hour
    long_window: 24.hours

  # Thresholds for state transitions
  thresholds:
    monitoring:
      min_signals: 1
      confidence_min: 0.50
      cooldown: 5.minutes

    suspect:
      min_signals: 2
      hard_signals: 1
      confidence_min: 0.70
      error_rate_spike: 0.30
      cooldown: 10.minutes

    quarantine:
      min_signals: 3
      hard_signals: 2
      confidence_min: 0.80
      multiple_providers: 2
      OR:
        hard_signals: 5
        window: medium
      cooldown: 30.minutes

    delist_check:
      min_quarantine_duration: 24.hours

    manual_review:
      consecutive_failures: 3
      inconclusive_probes: 5

  # Quarantine settings
  quarantine:
    initial_duration: 24.hours
    backoff_multiplier: 1.5
    max_duration: 7.days
    cooldown_period: 1.hour

  # Delist verification
  delist:
    retry_message_enabled: true
    rbl_query_enabled: true
```

```
    time_based_fallback: true
    check_interval: 1.hour
    time_based_threshold: 7.days

    rbl_lists:
      - zen.spamhaus.org
      - bl.spamcop.net
      - b.barracudacentral.org

  # Pool health thresholds
  pool_health:
    min_active_ips: 3
    critical_threshold: 0.4  # 40% usable

  # Notifications
  notifications:
    enabled: true
    aggregate_warnings: true

    channels:
      - type: slack
        webhook_url: "${BLACKLIST_SLACK_WEBHOOK}"
        events: ["quarantined", "pool_health_critical", "manual_review"]

      - type: email
        recipients: ["ops@example.com"]
        events: ["quarantined", "manual_review"]

      - type: webhook
        url: "${BLACKLIST_WEBHOOK_URL}"
        events: all

      # - type: jira
      #   enabled: false
      #   site: "https://company.atlassian.net"
      #   username: "${JIRA_USER}"
      #   api_token: "${JIRA_TOKEN}"
      #   project: "OPS"
      #   issue_type: "Incident"
      #   events: ["quarantined", "pool_health_critical"]

  # Data retention
  retention:
    signals: 90.days
    evidence: 180.days
    probes: 90.days
    audit_logs: 2.years
```

11.2 Rails Configuration Loader

```
# config/initializers/ip_blacklist.rb
module Postal
  class Application < Rails::Application

    config.ip_blacklist = ActiveSupport::OrderedOptions.new

    # Load from YAML
    yaml_config = YAML.load_file(
      Rails.root.join("config/postal/ip_blacklist.yml")
    )["ip_blacklist"]

    # Parse durations
    config.ip_blacklist.tap do |cfg|
      cfg.enabled = yaml_config["enabled"]

      cfg.monitoring = ActiveSupport::OrderedOptions.new
      cfg.monitoring.signal_capture = yaml_config.dig("monitoring",
"signal_capture")
      cfg.monitoring.short_window =
parse_duration(yaml_config.dig("monitoring", "short_window"))
      cfg.monitoring.medium_window =
parse_duration(yaml_config.dig("monitoring", "medium_window"))
      cfg.monitoring.long_window = parse_duration(yaml_config.dig("monitoring",
"long_window"))

      cfg.thresholds = deep_symbolize(yaml_config["thresholds"])
      cfg.quarantine = deep_symbolize(yaml_config["quarantine"])
      cfg.delist = deep_symbolize(yaml_config["delist"])
      cfg.pool_health = deep_symbolize(yaml_config["pool_health"])
      cfg.notifications = deep_symbolize(yaml_config["notifications"])
      cfg.retention = deep_symbolize(yaml_config["retention"])
    end

    private

    def parse_duration(value)
      case value
      when Integer then value.seconds
      when String then eval(value)  # "15.minutes" → 900
      else value
      end
    end

    def deep_symbolize(hash)
      hash.deep_symbolize_keys
    end

  end
end
```

11.3 Environment Variables

Supporto per override via ENV:

```
# .env
BLACKLIST_ENABLED=true
BLACKLIST_SLACK_WEBHOOK=https://hooks.slack.com/services/XXX/YYY/ZZZ
BLACKLIST_WEBHOOK_URL=https://internal-tool.example.com/api/alerts
BLACKLIST_MIN_QUARANTINE_HOURS=24
BLACKLIST_RBL_ENABLED=true
```

11.4 Web UI Configuration

Admin Panel: `/admin/blacklist_settings`

Permette di modificare:

- Thresholds (via form inputs)
- Pattern enable/disable
- Notification channels
- Manual override autorizzati

Pattern Management: `/admin/blacklist_patterns`

CRUD interface per gestire IPBlacklistPattern:

- Creare nuovi pattern custom
- Disabilitare pattern esistenti
- Testare regex con esempi
- Ordinare per priority

---

12. Observability

12.1 Metrics (Prometheus/StatsD)

12.1.1 Counters

```
# Signals captured
Postal.stats.increment("ip_blacklist.signals.captured",
  tags: { type: signal_type, provider: provider })

# State transitions
Postal.stats.increment("ip_blacklist.transitions",
  tags: { from: old_state, to: new_state })

# Quarantine events
Postal.stats.increment("ip_blacklist.quarantined",
  tags: { pool: pool_name })

# Delist attempts
```

```
Postal.stats.increment("ip_blacklist.delist_attempts",
  tags: { strategy: strategy_name, result: result_status })

# Notifications sent
Postal.stats.increment("ip_blacklist.notifications",
  tags: { channel: channel_type, severity: severity })
```

12.1.2 Gauges

```
# Active IPs per pool
Postal.stats.gauge("ip_blacklist.pool.active_ips",
  value: active_count, tags: { pool: pool_name })

# Quarantined IPs per pool
Postal.stats.gauge("ip_blacklist.pool.quarantined_ips",
  value: quarantined_count, tags: { pool: pool_name })

# Pool health ratio
Postal.stats.gauge("ip_blacklist.pool.health_ratio",
  value: health_ratio, tags: { pool: pool_name })

# Current confidence score per IP
Postal.stats.gauge("ip_blacklist.ip.confidence",
  value: confidence, tags: { ip: ip_address })
```

12.1.3 Histograms

```
# Quarantine duration
Postal.stats.histogram("ip_blacklist.quarantine_duration_seconds",
  value: duration_seconds, tags: { pool: pool_name })

# Delist verification duration
Postal.stats.histogram("ip_blacklist.delist_verification_seconds",
  value: verification_seconds, tags: { strategy: strategy_name })

# Signal confidence distribution
Postal.stats.histogram("ip_blacklist.signal_confidence",
  value: confidence, tags: { type: signal_type })
```

12.2 Logging

12.2.1 Structured Logging

```
# Signal detection
KLogger.tagged("IP_BLACKLIST", "DETECTION") do
  KLogger.info("Signal detected", {
```

```
    ip: ip_address.ipv4,
    ip_id: ip_address.id,
    type: signal_type,
    confidence: confidence,
    provider: provider,
    pattern: pattern_name,
    smtp_code: smtp_code
  })
end

# State transition
KLogger.tagged("IP_BLACKLIST", "TRANSITION") do
  KLogger.warn("IP quarantined", {
    ip: ip_address.ipv4,
    ip_id: ip_address.id,
    from: old_state,
    to: new_state,
    confidence: confidence,
    evidence_id: evidence.id,
    duration: quarantine_duration
  })
end

# Delist attempt
KLogger.tagged("IP_BLACKLIST", "DELIST") do
  KLogger.info("Delist verification completed", {
    ip: ip_address.ipv4,
    strategy: strategy_name,
    result: result_status,
    probe_id: probe_attempt.id
  })
end
```

12.2.2 Log Levels

- **DEBUG**: Pattern matching details, calculation steps
- **INFO**: Signals detected, successful delist, state back to active
- **WARN**: State transitions to suspect/monitoring, degraded pool health
- **ERROR**: Quarantine events, pool critical, notification failures
- **FATAL**: System errors, data corruption

12.3 Dashboard Panels

12.3.1 Overview Dashboard

URL: `/admin/ip_blacklist/dashboard`

Panels:

1. **Pool Health Overview**
    - Card per ogni pool con health status

- Active/Quarantined/Total IPs
- Trend ultimi 7 giorni

2. **Recent Signals**
   - Timeline grafico signals/hour
   - Breakdown per signal type
   - Top providers affected
3. **Active Quarantines**
   - Lista IP attualmente quarantined
   - Confidence score
   - Estimated release time
   - Quick actions (force delist, extend, review)
4. **Delist Queue**
   - IP in DELIST_CHECK
   - Next verification time
   - Last probe result

12.3.2 IP Detail View

URL: `/ip_addresses/:id/blacklist`

Sections:

1. **Current Status**
   - Status badge
   - Confidence score
   - Quarantine countdown (se applicable)
   - Quick actions
2. **Signal Timeline**
   - Graph segnali ultimi 30 giorni
   - Annotazioni per state changes
3. **Evidence History**
   - Tabella evidence records
   - Window, confidence, providers, recommendation
4. **Probe Attempts**
   - Tabella tentativi delist
   - Strategy, result, timestamp
5. **Audit Log**
   - Chronological log ogni evento

- Actor, reason, evidence snapshot

12.3.3 Pattern Analysis

URL: `/admin/blacklist_patterns`

Metrics per pattern:

- Hit count (last 7d, 30d)
- Avg confidence
- False positive rate (se disponibile)
- Provider distribution
- Enable/disable toggle
- Edit link

12.4 Alerts & Monitoring

12.4.1 Prometheus Alerting Rules

```
groups:
  - name: ip_blacklist
    interval: 1m
    rules:
      - alert: IPPoolCriticalHealth
        expr: ip_blacklist_pool_health_ratio < 0.4
        for: 5m
        labels:
          severity: critical
        annotations:
          summary: "IP Pool {{ $labels.pool }} has critical health"
          description: "Health ratio is {{ $value }}, below 40% threshold"

      - alert: HighQuarantineRate
        expr: rate(ip_blacklist_quarantined_total[1h]) > 2
        for: 10m
        labels:
          severity: warning
        annotations:
          summary: "High IP quarantine rate detected"
          description: "More than 2 IPs quarantined in the last hour"

      - alert: DelistFailureRate
        expr: |
          rate(ip_blacklist_delist_attempts_total{result="failed"}[6h])
          /
          rate(ip_blacklist_delist_attempts_total[6h])
          > 0.8
        for: 30m
        labels:
          severity: warning
        annotations:
```

```
          summary: "High delist failure rate"
          description: "80%+ of delist attempts failing"
```

12.4.2 Grafana Dashboards

Panels suggeriti:

1. **Pool Health Heatmap**
   - X-axis: Time
   - Y-axis: Pool names
   - Color: Health ratio
2. **Signal Rate Graph**
   - Stacked area: signal types
   - Per pool
3. **Quarantine Duration Distribution**
   - Histogram
   - Buckets: <1d, 1-2d, 2-4d, 4-7d, >7d
4. **Top Affected Providers**
   - Bar chart
   - Provider domain vs signal count

---

13. Testing Strategy

13.1 Unit Tests

13.1.1 Pattern Matching

```ruby
# spec/services/ip_blacklist/detector_service_spec.rb
require "rails_helper"

RSpec.describe IPBlacklist::DetectorService do

  let(:ip_address) { create(:ip_address) }
  let(:server) { create(:server) }

  describe "pattern matching" do

    it "detects Spamhaus RBL mentions" do
      result = described_class.call(
        ip_address: ip_address,
        server: server,
        smtp_code: 550,
        smtp_message: "550-5.7.1 Service unavailable; Client host
```

```
[203.0.113.42] blocked using zen.spamhaus.org",
        provider_domain: "gmail.com"
      )

      expect(result).to be_present
      expect(result.signal_type).to eq("hard_blacklist")
      expect(result.confidence).to be >= 0.90
      expect(result.metadata["rbl_detected"]).to include("zen.spamhaus.org")
    end

    it "distinguishes IP blocks from recipient blocks" do
      # IP block
      ip_block = described_class.call(
        smtp_code: 550,
        smtp_message: "550 IP address blocked due to spam"
      )

      # Recipient block
      recipient_block = described_class.call(
        smtp_code: 550,
        smtp_message: "550 Recipient address rejected: User unknown"
      )

      expect(ip_block.signal_type).to eq("hard_blacklist")
      expect(recipient_block).to be_nil  # excluded pattern
    end

    it "handles multi-language SMTP responses" do
      responses = [
        "550 IP bloccato per spam",  # Italian
        "550 IP bloqué pour spam",   # French
        "550 IP blockiert wegen Spam" # German
      ]

      responses.each do |response|
        result = described_class.call(smtp_message: response)
        expect(result.signal_type).to eq("hard_blacklist")
      end
    end

  end

end
```

13.1.2 Confidence Calculation

```
# spec/services/ip_blacklist/monitor_service_spec.rb
require "rails_helper"

RSpec.describe IPBlacklist::MonitorService do
```

```
  describe "confidence scoring" do

    it "calculates weighted average correctly" do
      ip = create(:ip_address)

      # Create signals with different types and confidence
      create(:ip_blacklist_signal,
        ip_address: ip,
        signal_type: "hard_blacklist",
        confidence: 0.95
      )
      create(:ip_blacklist_signal,
        ip_address: ip,
        signal_type: "soft_blacklist",
        confidence: 0.70
      )

      service = described_class.new
      confidence = service.send(:calculate_base_confidence,
ip.blacklist_signals)

      # hard (0.95 * 1.0) + soft (0.70 * 0.7) = 1.44 / 1.7 = 0.847
      expect(confidence).to be_within(0.01).of(0.85)
    end

    it "applies provider diversity boost" do
      ip = create(:ip_address)

      # 3 providers
      create(:ip_blacklist_signal, ip_address: ip, provider_domain:
"gmail.com")
      create(:ip_blacklist_signal, ip_address: ip, provider_domain:
"outlook.com")
      create(:ip_blacklist_signal, ip_address: ip, provider_domain:
"yahoo.com")

      service = described_class.new
      boost = service.send(:diversity_boost, ip.blacklist_signals)

      expect(boost).to eq(1.30)  # 3 providers → 30% boost
    end

  end

end
```

13.1.3 State Machine

```
# spec/services/ip_blacklist/quarantine_service_spec.rb
require "rails_helper"
```

```ruby
RSpec.describe IPBlacklist::QuarantineService do

  describe "state transitions" do

    it "transitions ACTIVE → SUSPECT with threshold signals" do
      ip = create(:ip_address)
      evidence = create(:ip_blacklist_evidence,
        ip_address: ip,
        aggregate_confidence: 0.75
      )

      result = described_class.transition_to(
        ip_address: ip,
        new_state: "suspect",
        evidence: evidence,
        actor: "System"
      )

      expect(result).to be true
      expect(ip.reload.blacklist_status.status).to eq("suspect")
    end

    it "prevents flapping with cooldown" do
      ip = create(:ip_address)
      create(:ip_address_status,
        ip_address: ip,
        status: "monitoring",
        status_changed_at: 2.minutes.ago
      )

      # Try to transition before cooldown (5 min)
      result = described_class.transition_to(
        ip_address: ip,
        new_state: "suspect",
        evidence: nil,
        actor: "System"
      )

      expect(result).to be false
      expect(ip.reload.blacklist_status.status).to eq("monitoring")
    end

    it "increments quarantine_count on quarantine" do
      ip = create(:ip_address)
      status = create(:ip_address_status,
        ip_address: ip,
        quarantine_count: 2
      )

      described_class.transition_to(
        ip_address: ip,
        new_state: "quarantined",
        evidence: nil,
        actor: "System"
```

```
      )

      expect(status.reload.quarantine_count).to eq(3)
    end

  end

end
```

13.2 Integration Tests

```
# spec/integration/ip_blacklist_flow_spec.rb
require "rails_helper"

RSpec.describe "IP Blacklist Full Flow" do

  it "quarantines IP after blacklist signals and notifies" do
    # Setup
    ip_pool = create(:ip_pool)
    ip = create(:ip_address, ip_pool: ip_pool)
    server = create(:server, ip_pool: ip_pool)

    # Simulate 3 failed deliveries with blacklist errors
    3.times do |i|
      delivery_data = {
        ip_address: ip,
        server: server,
        smtp_code: 550,
        smtp_message: "550-5.7.1 blocked using zen.spamhaus.org",
        provider_domain: "gmail.com"
      }

      IPBlacklist::DetectorService.call(delivery_data)
    end

    # Run monitor job
    IPBlacklist::MonitorService.call

    # Verify IP quarantined
    expect(ip.reload.blacklist_status.status).to eq("quarantined")
    expect(ip.blacklist_status.confidence_score).to be > 0.80

    # Verify notification sent
    expect(ActionMailer::Base.deliveries).not_to be_empty
    email = ActionMailer::Base.deliveries.last
    expect(email.subject).to include("quarantined")
  end

  it "auto-delists IP after successful retry" do
    # Setup quarantined IP
    ip = create(:ip_address)
```

```
      create(:ip_address_status,
        ip_address: ip,
        status: "quarantined",
        last_quarantined_at: 25.hours.ago
      )

      # Transition to delist_check
      IPBlacklist::MonitorService.call
      expect(ip.reload.blacklist_status.status).to eq("delist_check")

      # Mock successful delivery
      allow_any_instance_of(IPBlacklist::DelistStrategies::RetryOriginalMessage)
        .to receive(:execute).and_return(:success)

      # Run delist job
      IPBlacklist::DelistVerifierService.call

      # Verify IP back to monitoring
      expect(ip.reload.blacklist_status.status).to eq("monitoring")

      # Verify notification
      expect(ActionMailer::Base.deliveries.last.subject).to include("delisted")
    end

    it "maintains pool health during mass quarantine" do
      pool = create(:ip_pool)
      ips = create_list(:ip_address, 5, ip_pool: pool)

      # Quarantine 2 IPs
      ips.take(2).each do |ip|
        IPBlacklist::QuarantineService.transition_to(
          ip_address: ip,
          new_state: "quarantined",
          evidence: nil,
          actor: "System"
        )
      end

      # Check metrics
      metrics = pool.health_metrics
      expect(metrics[:usable]).to eq(3)
      expect(metrics[:health_ratio]).to be_within(0.01).of(0.6)
      expect(metrics[:status]).to eq("degraded")

      # Selection still works
      selected = pool.ip_addresses.usable_for_sending.first
      expect(selected).to be_present
      expect(selected.quarantined?).to be false
    end

  end
```

13.3 Load Testing

```ruby
# spec/performance/ip_blacklist_performance_spec.rb
require "rails_helper"

RSpec.describe "IP Blacklist Performance" do

  it "handles 1000 signals/min without degradation" do
    ip = create(:ip_address)
    server = create(:server)

    benchmark = Benchmark.measure do
      1000.times do
        IPBlacklist::DetectorService.call(
          ip_address: ip,
          server: server,
          smtp_code: 550,
          smtp_message: "550 blocked",
          provider_domain: "example.com"
        )
      end
    end

    # Should complete in < 10 seconds
    expect(benchmark.real).to be < 10

    # Should create all signals
    expect(IPBlacklistSignal.count).to eq(1000)
  end

  it "processes backlog after downtime" do
    # Create 10K old signals
    create_list(:ip_blacklist_signal, 10_000,
      detected_at: 1.hour.ago
    )

    benchmark = Benchmark.measure do
      IPBlacklist::MonitorService.call
    end

    # Should complete in < 30 seconds
    expect(benchmark.real).to be < 30
  end

end
```

---

14. Rollout Plan

14.1 Phase 1: Observation Mode (2 settimane)

Obiettivo: Validare detection accuracy senza impatto su invii.

Azioni:

1. Deploy signal capture (DetectorService)
2. Disabilita state transitions (solo logging)
3. Popola IPBlacklistPattern con pattern predefiniti
4. Configura notification webhook per monitoraggio

Validazione:

- Review giornaliero signals catturati
- False positive rate < 5%
- Coverage pattern: almeno 90% errori classificati
- Performance: latency < 50ms per signal capture

Criteri Success:

- Nessun impatto su delivery pipeline
- Pattern matching accurato su sample 100 errori
- Stakeholder approval su sample signals

14.2 Phase 2: Monitoring & Alerts (2 settimane)

Obiettivo: Attivare monitoring e notifiche senza quarantine.

Azioni:

1. Abilita MonitorService (background job ogni 5 min)
2. Attiva transizioni ACTIVE → MONITORING → SUSPECT
3. Configura notifiche Slack/Email per eventi WARNING
4. Deploy dashboard UI

Validazione:

- Notifiche ricevute entro 15 min da spike
- Confidence score correlato con verità ground truth
- Dashboard accessibile e informativo

Criteri Success:

- Zero falsi negativi (tutti i blacklist reali rilevati)
- Notifiche utili (no alert fatigue)
- Team ops trained su dashboard

14.3 Phase 3: Quarantine Enforcement (1 settimana)

Obiettivo: Attivare quarantine automatica con monitoraggio stretto.

Azioni:

1. Abilita transizione SUSPECT → QUARANTINED

2. Deploy enforcement in IP selection logic
3. Configura notifiche ERROR per quarantine
4. Preparare pool fallback per emergenze

Validazione:

- IP quarantined effettivamente esclusi da selezione
- Pool health monitored e stabile
- Nessun outage causato da quarantine

Rollback Plan:

- Disable quarantine via config flag
- Force all quarantined IPs back to ACTIVE
- Investigate root cause

Criteri Success:

- Almeno 1 IP correttamente quarantined e verificato
- Nessun falso negativo
- Pool health sempre > 50%

14.4 Phase 4: Auto-Delist (1-2 settimane)

Obiettivo: Automazione completa del ciclo.

Azioni:

1. Abilita DelistVerifierService
2. Attiva retry strategy
3. Attiva RBL query strategy (se disponibile)
4. Monitora success rate delist

Validazione:

- Almeno 1 delist automatico riuscito
- Nessun IP delistato erroneamente
- Probe attempts tracked e auditati

Criteri Success:

- Delist success rate > 70%
- Tempo medio quarantine < 48h
- Zero manual interventions necessari (eccetto manual_review)

14.5 Phase 5: Tuning & Optimization (ongoing)

Azioni:

- Tune thresholds basati su dati reali
- Aggiungere pattern custom per provider specifici
- Ottimizzare performance query DB

- Implementare ML per anomaly detection (futuro)

Metrics da Ottimizzare:

- False positive rate < 2%
- Quarantine duration avg < 36h
- Delist success rate > 85%
- Notification relevance score (feedback operatori)

---

15. Security & Access Control

15.1 RBAC (Role-Based Access Control)

15.1.1 Roles

Role	Permissions
Admin	Full access: view, edit patterns, force state changes, override
DeliverabilityOps	View all, force state changes, manual review decisions
DeliverabilityViewer	Read-only: view status, signals, history
ServerOwner	View own servers' IP status only
Regular User	No access

15.1.2 Pundit Policies

```
# app/policies/ip_address_status_policy.rb
class IPAddressStatusPolicy < ApplicationPolicy

  def show?
    user.admin? ||
    user.deliverability_ops? ||
    user.deliverability_viewer? ||
    user_owns_server?
  end

  def update?
    user.admin? || user.deliverability_ops?
  end

  def force_quarantine?
    user.admin?
  end

  def force_delist?
    user.admin? || user.deliverability_manager?
  end
```

```
  private

  def user_owns_server?
    record.ip_address.servers.any? { |s| s.organization.users.include?(user) }
  end

end
```

15.2 Audit Trail

Ogni azione sensibile viene loggata:

```
# After any state change
IPBlacklistAuditLog.create!(
  ip_address: ip,
  event_type: "state_change",
  from_status: old_state,
  to_status: new_state,
  actor_id: current_user&.id,
  actor_type: current_user ? "User" : "System",
  reason: reason,
  evidence_snapshot: evidence_data,
  occurred_at: Time.current
)
```

Immutability: Audit logs non possono essere modificati/cancellati (solo INSERT).

15.3 Rate Limiting

Previene abusi di override manuali:

```
# app/middleware/ip_blacklist_rate_limiter.rb
class IPBlacklistRateLimiter

  MAX_OVERRIDES_PER_HOUR = 10

  def call(user, action)
    key = "ip_blacklist:overrides:#{user.id}:#{Date.current.to_s}"
    count = Rails.cache.increment(key, 1, expires_in: 1.hour)

    if count > MAX_OVERRIDES_PER_HOUR
      raise RateLimitExceeded, "Maximum #{MAX_OVERRIDES_PER_HOUR} overrides per
hour exceeded"
    end
  end

end
```

15.4 API Access

Se esposto via API:

```
# API authentication via API key
class API::V2::IPBlacklistController < API::BaseController

  before_action :authenticate_api_key!
  before_action :require_deliverability_scope!

  def status
    ip = IPAddress.find(params[:id])
    authorize ip.blacklist_status

    render json: IPBlacklistSerializer.new(ip).as_json
  end

end
```

Scopes richiesti:

- `ip_blacklist:read` - View status
- `ip_blacklist:write` - Force state changes

15.5 Sensitive Data Handling

PII Protection:

- Signals NON contengono recipient email addresses (solo provider domain)
- SMTP messages troncati a 1000 chars (evitare full email content)
- Metadata non contiene PII

Credentials:

- Webhook URLs, JIRA tokens, etc. in ENV vars (non in DB)
- Encrypted at rest se stored in DB (Rails encrypted attributes)

---

16. Failure Modes & Recovery

16.1 Circuit Breakers

16.1.1 Detection Service Circuit Breaker

```
# Se detection service fallisce troppo spesso, disabilitare temporaneamente
class IPBlacklistCircuitBreaker

  FAILURE_THRESHOLD = 10  # 10 errori consecutivi
  TIMEOUT_DURATION = 5.minutes
```

```
  def self.call(service_name)
    if circuit_open?(service_name)
      KLogger.warn("Circuit breaker open for #{service_name}, skipping")
      return nil
    end

    begin
      yield
      record_success(service_name)
    rescue => e
      record_failure(service_name, e)
      raise if should_raise?(service_name)
    end
  end

  private

  def self.circuit_open?(service_name)
    failures = Rails.cache.read("circuit:#{service_name}:failures") || 0
    failures >= FAILURE_THRESHOLD
  end

  def self.record_failure(service_name, error)
    failures = Rails.cache.increment("circuit:#{service_name}:failures", 1,
expires_in: TIMEOUT_DURATION)

    if failures >= FAILURE_THRESHOLD
      KLogger.error("Circuit breaker opened for #{service_name}", { error:
error.message })
      IPBlacklist::NotificationService.call(
        event: "circuit_breaker_open",
        severity: "critical",
        data: { service: service_name, error: error.message }
      )
    end
  end

  def self.record_success(service_name)
    Rails.cache.delete("circuit:#{service_name}:failures")
  end

end

# Usage
IPBlacklistCircuitBreaker.call("detector") do
  IPBlacklist::DetectorService.call(data)
end
```

16.1.2 Notification Circuit Breaker

Se notifiche falliscono, non bloccare operazioni critiche:

```
def send_to_channel(channel, payload)
  Timeout.timeout(10) do
    # ... send logic
  end
rescue => e
  # Log ma non raise
  KLogger.error("Notification failed", { channel: channel[:type], error:
e.message })

  # Store for retry
  store_failed_notification(channel, payload, e)
end
```

16.2 Idempotency

16.2.1 Signal Creation

Evitare duplicati:

```
# Use unique constraint on (delivery_id, ip_address_id)
add_index :ip_blacklist_signals, [:delivery_id, :ip_address_id], unique: true,
name: "unique_signal_per_delivery"

# In service
def create_signal
  IPBlacklistSignal.create!(attrs)
rescue ActiveRecord::RecordNotUnique
  # Already processed, skip
  KLogger.debug("Duplicate signal skipped", { delivery_id: @delivery_id })
  nil
end
```

16.2.2 State Transitions

Prevent duplicate transitions:

```
def execute_transition
  # Lock record
  @ip_address.with_lock do
    current_status = @ip_address.blacklist_status.status

    # Idempotent: già nello stato target
    if current_status == @new_state
      KLogger.debug("Already in state #{@new_state}, skipping")
      return true
    end
```

```
    # ... proceed with transition
  end
end
```

16.3 Recovery Procedures

16.3.1 Rake Tasks

```
# lib/tasks/ip_blacklist.rake
namespace :ip_blacklist do

  desc "Force delist an IP"
  task :force_delist, [:ip_address_id] => :environment do |t, args|
    ip = IPAddress.find(args[:ip_address_id])

    result = IPBlacklist::QuarantineService.transition_to(
      ip_address: ip,
      new_state: "active",
      evidence: nil,
      actor: "RakeTask",
      reason: "Manual force delist via rake task"
    )

    puts result ? "✓ IP delisted" : "✗ Failed to delist"
  end

  desc "Reset all IPs in pool to active"
  task :reset_pool, [:pool_id] => :environment do |t, args|
    pool = IPPool.find(args[:pool_id])

    pool.ip_addresses.each do |ip|
      IPBlacklist::QuarantineService.transition_to(
        ip_address: ip,
        new_state: "active",
        evidence: nil,
        actor: "RakeTask",
        reason: "Pool reset via rake task"
      )
    end

    puts "✓ Pool #{pool.name} reset"
  end

  desc "Reprocess signals for time range"
  task :reprocess_signals, [:start_time, :end_time] => :environment do |t,
args|
    start_time = Time.parse(args[:start_time])
    end_time = Time.parse(args[:end_time])

    signals = IPBlacklistSignal.where(detected_at: start_time..end_time)
```

```
    signals.find_each do |signal|
      # Re-run monitor for this IP
      # ... implementation
    end

    puts "✓ Reprocessed #{signals.count} signals"
  end

  desc "Disable system temporarily"
  task :disable => :environment do
    Rails.cache.write("ip_blacklist:disabled", true, expires_in: 1.hour)
    puts "✓ IP Blacklist system disabled for 1 hour"
  end

  desc "Enable system"
  task :enable => :environment do
    Rails.cache.delete("ip_blacklist:disabled")
    puts "✓ IP Blacklist system enabled"
  end

end
```

16.3.2 Emergency Procedures

Scenario 1: False mass quarantine

```
# Disable system
rake ip_blacklist:disable

# Reset affected pool
rake ip_blacklist:reset_pool[5]

# Investigate root cause
# Fix pattern or threshold

# Re-enable
rake ip_blacklist:enable
```

Scenario 2: Pool health critical

```
# Quick: Force delist safe IPs
rake ip_blacklist:force_delist[42]
rake ip_blacklist:force_delist[43]

# Or: Use fallback pool temporarily
# Update server config to point to backup pool
```

Scenario 3: Notification storm

```
# Disable notifications
Rails.cache.write("ip_blacklist:notifications:disabled", true)

# Clear aggregation queue
# Fix root cause

# Re-enable notifications
Rails.cache.delete("ip_blacklist:notifications:disabled")
```

16.4 Data Corruption Recovery

Backup Strategy:

- Daily backup audit_logs (critical for compliance)
- Weekly backup signals/evidence (può essere ricostruito)
- Real-time backup status records (critical per operations)

Recovery:

```
# Rebuild evidence from signals
rake ip_blacklist:rebuild_evidence[ip_address_id, start_date, end_date]

# Recalculate confidence scores
rake ip_blacklist:recalculate_confidence[ip_address_id]
```

---

17. Compliance & Retention

17.1 Data Retention Policy

```
retention:
  ip_blacklist_signals: 90.days       # transient, può essere purgato
  ip_blacklist_evidence: 180.days     # analytical, retention estesa
  ip_probe_attempts: 90.days          # operational, retention media
  ip_blacklist_audit_logs: 2.years    # compliance, retention lunga
```

Rationale:

- **Signals**: High-volume, valore diminuisce rapidamente → retention breve
- **Evidence**: Utile per trend analysis → retention media
- **Audit logs**: Compliance requirement, immutable → retention lunga

17.2 GDPR Considerations

Data Processed:

- IP addresses (not PII under GDPR)
- Provider domains (not PII)
- SMTP error messages (may contain email addresses in quotes)
- Operator usernames (PII - legit interest for audit)

Lawful Basis:

- Legitimate interest: Protezione infrastruttura email, prevenzione spam
- Contract performance: Delivery service richiede reputazione management

Data Subject Rights:

- Right to access: API/UI per view status proprio server
- Right to rectification: Manual override mechanism
- Right to erasure: Not applicable (IP addresses non sono PII)
- Right to explanation: Evidence snapshot fornisce rationale decisioni

Recommended Practices:

- Anonimizzare SMTP messages dopo retention period
- Pseudonimizzare operator IDs in audit logs after 6 months
- Document legitimate interest assessment

17.3 SOC 2 / Audit Requirements

Controls da Implementare:

1. **Access Control**: RBAC con least privilege
2. **Audit Logging**: Immutable logs di ogni cambio stato
3. **Change Management**: Versioning configuration, approval workflow pattern changes
4. **Incident Response**: Playbook per false positives, mass quarantine, outages
5. **Business Continuity**: Fallback pools, manual override capability

Audit Evidence:

- IPBlacklistAuditLog records (chi, cosa, quando, perché)
- Configuration change history (git commits)
- Incident reports (post-mortem docs)
- Regular access reviews (quarterly)

17.4 Archival Strategy

Cold Storage per Audit Logs >1 year:

```
# Scheduled job: export to S3/archive DB
class ArchiveOldAuditLogsJob

  def perform
    cutoff = 1.year.ago

    logs = IPBlacklistAuditLog.where("occurred_at < ?", cutoff)
```

```
    # Export to JSON
    archive_file = "/tmp/audit_logs_#{Date.current}.jsonl"
    File.open(archive_file, "w") do |f|
      logs.find_each do |log|
        f.puts log.to_json
      end
    end

    # Upload to S3
    s3_client.upload_file(archive_file, "postal-audit-archives",
"ip_blacklist/#{Date.current}.jsonl.gz")

    # Delete from DB (dopo conferma upload)
    logs.delete_all

    KLogger.info("Archived #{logs.count} audit logs to S3")
  end

end
```

---

18. Open Questions & Future Enhancements

18.1 Open Questions

18.1.1 Technical

1. **Machine Learning Integration**
   - **Q**: Sostituire threshold fissi con anomaly detection ML?
   - **Consideration**: Richiede training data, rischio false positives, explainability
   - **Recommendation**: Evaluate dopo 6 mesi dati reali
2. **Real-time vs Batch Processing**
   - **Q**: Signal aggregation ogni 5 min sufficiente o serve real-time streaming?
   - **Consideration**: Complessità architettura (Kafka, stream processing) vs latency gain
   - **Recommendation**: Iniziare batch, valutare real-time se richiesto SLA < 1 min
3. **Multi-region Blacklists**
   - **Q**: IP può essere blacklistato solo in certe regioni geografiche?
   - **Consideration**: Complessità tracking, provider geo-specifici
   - **Recommendation**: Fuori scope v1, raccogliere dati regionali

18.1.2 Operational

1. **SLA per Delist**
   - **Q**: Quale è il tempo massimo accettabile di quarantena?

- **Consideration**: Impatto business, urgenza recovery
- **Recommendation**: Definire SLA con stakeholder (proposta: 48h avg, 7d max)

2. **Provider Whitelist**

- **Q**: Alcuni IP hanno whitelist agreements con provider (Gmail, Outlook). Come integrarli?
- **Consideration**: API Google Postmaster Tools, Microsoft SNDS
- **Recommendation**: Phase 2 enhancement

3. **Cost of RBL Queries**

- **Q**: Alcuni RBL hanno rate limits o costi. Budget disponibile?
- **Consideration**: Query volume, alternative free RBLs
- **Recommendation**: Iniziare con free tier, valutare paid se necessario

18.2 Future Enhancements

18.2.1 Short-term (3-6 mesi)

1. **Enhanced Provider Intelligence**

- Integrazione Google Postmaster Tools API
- Integrazione Microsoft SNDS
- Yahoo FBL processing
- Provider-specific thresholds (Gmail più strict, altri relaxed)

2. **Advanced Confidence Algorithms**

- Bayesian inference per confidence calculation
- Time-series anomaly detection
- Seasonal pattern recognition (volume spikes normali vs anomali)

3. **Improved Delist Verification**

- Seed email addresses per probe testing
- Warming protocol post-delist (gradual volume ramp)
- Multi-step verification (probe + RBL + time)

4. **Richer Notifications**

- Slack interactive buttons (Approve Delist, Extend Quarantine, etc.)
- PagerDuty integration per critical alerts
- SMS notifications per emergenze

18.2.2 Medium-term (6-12 mesi)

1. **Machine Learning Models**

- Supervised learning per pattern classification
- Unsupervised anomaly detection per nuovi pattern
- Predictive model: likelihood IP sarà blacklistato (preventive action)

2. **IP Reputation Scoring**
    - Aggregare metriche oltre blacklist: bounce rate, complaint rate, engagement
    - Score 0-100 per ogni IP
    - Dynamic priority allocation basata su reputation
3. **Automated Remediation**
    - Auto-submit delist requests a RBL (quando supportato)
    - Auto-warming per IP nuovi
    - Self-healing pool management (auto-provisioning IP replacement)
4. **Advanced Analytics**
    - Campaign correlation: quali campagne causano blacklist
    - Content analysis: pattern nei messaggi che triggerano spam filters
    - Recipient domain profiling: quali domini più strict

18.2.3 Long-term (12+ mesi)

1. **Predictive Blacklist Prevention**
    - Pre-flight checks prima di inviare campaign
    - Content scoring: likelihood messaggio sarà marcato spam
    - Recipient validation: bounce/complaint prediction
2. **Multi-MTA Coordination**
    - Se Postal scala a multiple instance, share blacklist intelligence
    - Centralized IP reputation database
    - Cross-instance learning
3. **Blockchain Audit Trail**
    - Immutable, distributed audit log
    - Compliance proof per auditor
    - Cryptographic verification state changes
4. **AI-Powered Decision Support**
    - Chatbot per operator: "Why was IP X quarantined?"
    - Recommendation engine: "Best action for IP Y?"
    - Automated root cause analysis

18.3 Research Topics

1. **Federated Learning per Blacklist Detection**
    - Multiple Postal instance share learning senza condividere dati
    - Privacy-preserving pattern discovery
2. **Graph Analysis per Reputation Networks**

    - Model sender reputation come graph
    - Propagate reputation tra IP dello stesso pool

3. **Quantum-Resistant Audit Logs**
    - Future-proof audit trail contro quantum computing

Conclusioni

Questo documento definisce un sistema completo per la gestione automatica delle blacklist IP in Postal, bilanciando:

- **Affidabilità**: Zero falsi negativi, detection accurata
- **Automazione**: Minimizzare intervento umano mantenendo controllo
- **Osservabilità**: Metriche, dashboard, audit trail completo
- **Scalabilità**: Gestire volumi elevati senza degradazione
- **Compliance**: Retention policy, GDPR, audit requirements

Next Steps:

1. Review tecnico con team Postal
2. Approval stakeholder
3. Prioritizzazione fasi rollout
4. Inizio implementazione Phase 1

Domande o Feedback: Contattare il team deliverability ops.

Document Version: 1.0
Last Updated: 2026-01-26
Status: Draft - Pending Review